



School of Informatics and IT
Diploma in Big Data & Analytics

Major Project Final Report

Project Title:

AI-Enhanced Learning Insights and Personalized Student Support

Submitted By:

Sing Yan Ling, 2300984H

Supervisor:

Miss Esther Lee, Lecturer, School of Informatics & IT

Declaration of Originality

I am the originator of this work and I have appropriately acknowledged all other original sources used as my reference for this work.

I understand that Plagiarism is the act of taking and using the whole or any part of another person's work, including work generated by AI, and presenting it as my own.

I understand that Plagiarism is an academic offence and if I am found to have committed or abetted the offence of plagiarism in relation to this submitted work, disciplinary action will be enforced.

☒ I Agree (Please Tick ✓)

Student Name (Matric Number) : Sing Yan Ling (2300984H)

Submission Date : 8 August 2025

Table of Contents

Major Project Final Report.....	0
Table of Contents.....	2
Acknowledgements.....	2
Abstract.....	4
Chapter 1: Introduction.....	1
Background of the Project.....	1
Problems Identified.....	1
Objectives of the Project.....	2
Scope of the Project.....	2
System Architecture Diagram.....	3
Chapter 2: Configuration of Synthetic Data Generation.....	4
Background and Justification.....	4
Scalability and Adaptation.....	6
Project Implementation.....	7
Chapter 3: Backend-as-a-Service: SQL Database + API Infrastructure.....	10
Background and Justification.....	10
Scalability and Adaptation.....	11
Project Implementation.....	12
Chapter 4: Python Web Development with Streamlit Framework.....	17
Background and Justification.....	17
Scalability and Adaptation.....	20
Project Implementation.....	20
Chapter 5: Generative AI prompting + RAG.....	21
Background and Justification.....	21
Advanced Generative AI Techniques.....	25
Scalability and Adaptation.....	26
Project Implementation.....	27
Chapter 6: Data Visualization and Predictive Modelling.....	38
Background and Justification.....	38
Scalability and Adaptability.....	38
Project Implementation.....	39
Chapter 7: Evaluation/Review.....	46
Chapter 8: Reflection.....	46
Lessons Learnt and Insights Gained.....	47
References/ Bibliography.....	48
Appendix A1: Terms of Reference.....	53
Appendix A2: Weekly Progress Report.....	56

Acknowledgements

I would like to thank Miss Esther Lee, my MP supervisor, for her guidance and support throughout the entire duration of this project. Her advice and constructive feedback greatly contributed to shaping the direction and product of my work. I appreciate her patience and encouragement, which motivated me to overcome challenges and achieve the project goals successfully.

Abstract

This project aims to develop a unified student learning analytics platform to provide real-time insights into student engagement and performance, addressing current limitations in Learning Management Systems (LMS). To address this, synthetic data was generated to simulate quiz, test and engagement metrics for model learning and dashboard visualization. Machine learning, specifically K-Means clustering, is done to group students by their academic performance, allowing for targeted academic support. Additionally, a Generative AI model was implemented to provide personalized feedback and explanations for mistakes. Interactive Tableau dashboards connected to a cloud-based SQL database ensured scalability and adaptability for evolving datasets. Key findings include key findings of improved visibility in student's metrics and learning gaps and enhanced personalization for feedback.

Chapter 1: Introduction

Background of the Project

Learning Management System (LMS) is an online platform where students can interact with various online learning materials such as quizzes, e-lectures and study material uploaded by their lecturers. However, these resources are spread across the website, lacking an integrated view and lecturers rely on manual tracking methods to identify at-risk students and weak topic areas, leading to inefficiencies and difficulty in monitoring student progress and performance.

This project aims to develop a unified analytics platform that provides real-time insights into student engagement and performance through integrated dashboards and intelligent feedback mechanisms.

Problems Identified

Lack of Visibility into Student Engagement & Performance

- Without real-time insights, lecturers do not know how students are engaging with learning materials such as quizzes and video lectures. There isn't a unified platform where the quiz attempts, test scores, common mistakes/weak topic/areas and total watch time are consolidated, resulting in lecturers struggling to identify students who needs help early and provide help

No Automated Way to Identify Common Mistakes & Weak Areas

- There are common topic/questions that students are bound to make repeated mistakes in quizzes & tests and currently lecturers have to manually analyze those performance trends, making it time-consuming & inefficient to detect those common weak topics as it tends to shift overtime

Generic Learning Recommendations Instead of Personalized Support

- Currently, regardless of their learning gaps and areas of weakness, students are receiving the same set of study materials. This makes improvement for students inefficient as those recommended studying materials are not personalized to adapt to each student's needs to bridge their learning gaps efficiently.

No AI-Powered Student Support for Learning Difficulties

- The existing system does not provide automated explanations for student's mistakes and lecturers are not able to provide individualized feedback for all of their students given if they have a large number of students. Some students may also hesitate to ask their lecturers for help when struggling with certain concepts, resulting in a larger studying bridge in the long run.

Manual & Time-Consuming Academic Monitoring for Lecturers

- With the current system, lecturers have to manually track student's performance and engagement across multiple tabs. This is inefficient and can be time consuming which may result in untimely intervention.

Objectives of the Project

- To design and build a centralised learning analytics platform that includes student engagement and performance data for students and educators alike.
- To leverage advanced generative AI techniques, such as retrieval-augmented generation and multi-modal generative AI models to deliver customised feedback and a wide range of learning materials in a variety of formats.
- To automatically identify and facilitate the exploration of common mistakes, weak topics, and low-performing students who require timely intervention.
- To improve the learning experience for students and teaching experience for educators by applying descriptive, predictive, and prescriptive analytics through real-time dashboards and AI summaries.
- To develop a modular, scalable and adaptable system architecture that integrates with Temasek Polytechnic's student information systems and supports the fast-paced, rapidly changing field of analytics and AI.

Scope of the Project

- **Generative AI–Powered Synthetic Data Generation:** Implement synthetic datasets using generative AI techniques to simulate realistic learning materials, student performance data, and engagement metrics for development and testing.
- **Backend-as-a-Service: Supabase:** Utilize Supabase to manage authentication, store structured and unstructured data, and support semantic search capabilities for AI-powered retrieval.
- **Python Web Development in Streamlit:** Develop an interactive and user-friendly web application interface entirely in Python, enabling quick prototyping and seamless integration with backend and AI components.

- **Generative AI Features:** Integrate large language model–driven tools, including basic Retrieval-Augmented Generation (RAG), personalized feedback, AI-generated flashcards, and multimodal learning aids.
- **Data Visualization and Predictive Modelling:** Implement dashboards and visual analytics for educators, along with predictive modelling to identify trends in student performance and engagement.

System Architecture Diagram

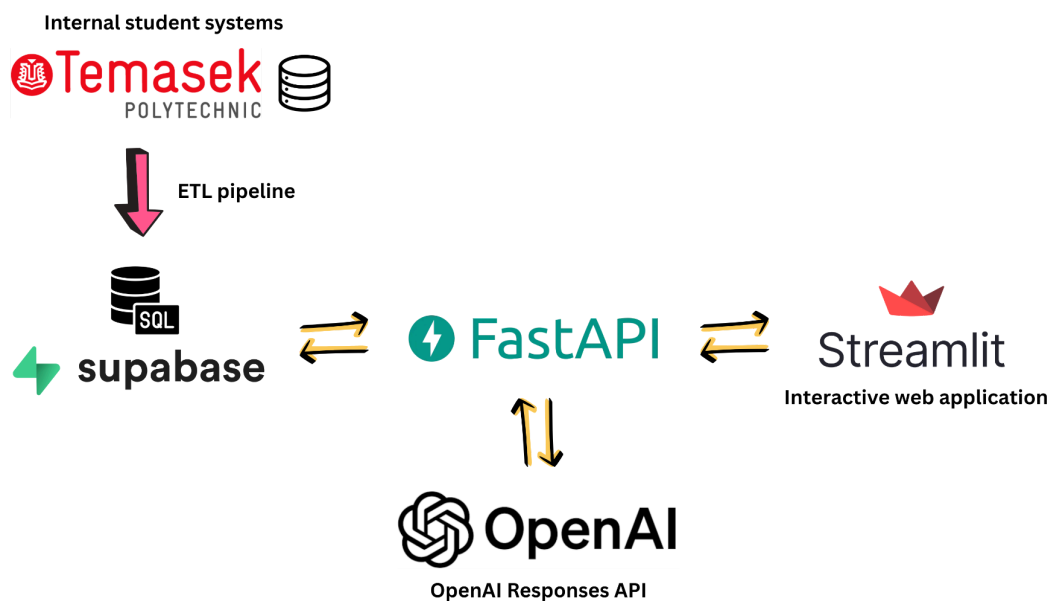


Figure 1.1: High-level system architecture diagram

Figure 1.1 represents the high-level system architecture of the application. Each component in the system plays a distinct role in ingesting, storing, serving and making data available to end users. The design is modular and each component is designed to be as portable, scalable and adaptable as possible. Individual components can be changed out for suitable alternatives without significant adjustments needed to the system architecture or efforts to rewrite the code.

Temasek Polytechnic's (TP's) internal student information systems store valuable, granular information about the quizzes and tests each student takes for each module. While the exact structure and format of the data is not known to the author, a hypothetical student information system would store personal and academic information about each student. Automated extract, transform, and load processes would be developed to take valuable data

out of these systems and store them on the relational database. However, this project will utilise synthetic data in place of TP's student information systems.

Supabase acts as the database layer, often termed the back-end. It stores student data and supports the efficient ingestion, storage, and querying of data by upstream components.

FastAPI acts as the backend integration layer, as it is the central hub that connects the front-end to various back-end services. This is especially essential in an application that has multiple back-end services like OpenAI API and Supabase. While FastAPI is not technically a necessity as Streamlit can directly connect to Supabase and OpenAI, it allows for separation of concerns - enabling better security for sensitive API keys by not exposing them to the front-end and flexibility by enabling API implementations to be modified independent of the front-end Streamlit code.

OpenAI provides state-of-the-art artificial intelligence (AI) models which power many of this application's most unique features. It combines textual prompts with a wide variety of structured and unstructured data from Supabase's database to produce results in a wide variety of formats -- text, audio, and video.

Streamlit is the front-end application which users directly interact with. It presents a web interface for both educators and students to explore, interact with, and learn more about their work. Streamlit interacts with FastAPI through pre-defined HTTP requests, which allows it to access and interact with the database and AI models.

Chapter 2: Configuration of Synthetic Data Generation

Tools used: OpenAI GPT 4.1 mini (OpenAI API), Python

Background and Justification

Synthetic data is widely defined as artificial data that is statistically similar to real-world data. It is often used as a placeholder for real data in machine learning and software development. This is especially so in contexts where data is scarce, insufficient in quantity, or challenging to obtain or use for privacy or legal reasons (Lu et al., 2025). Synthetic data generation enables researchers to generate data that is similar in format, structure, and distribution to real data, while mitigating the challenges of collecting and using real data (Personal Data Protection Commission, 2024).

The nature of synthetic data denotes that it is generated by a software program, which takes in an input and returns the generated data as output. There are two key categories of methods used for synthetic data generation: statistical and deep-learning methods.

Traditionally, the statistical method involves sampling the underlying distribution of real-world observed data and then generating artificial samples based on this known sampling distribution (Goyal & Mahmoud, 2024). This is usually done by computing the statistical parameters (such as mean, variance, and kernel density estimation) of real-world data and utilising random sampling or probability distribution functions to generate new samples (Javergard et al., 2025). In doing so, the generated data is statistically similar to real-world data while not being identical or directly related to any actual records.

On the other hand, the meteoric rise of deep-learning infrastructure and algorithms in the last decade has also led to an increasing popularity of deep-learning based synthetic data generation methods. These methods are broadly known as generative AI methods. For instance, the generative adversarial network (GAN) was introduced in 2014. It consists of two neural networks, a generator and discriminator, which work in opposition to each other. As training progresses, the generator learns to generate realistic data, while the discriminator learns to classify the data as artificially generated or genuine (Goodfellow, 2014). This ingenious solution led to much interest in deep-learning based data generation methods.

In 2018, researchers at OpenAI introduced the generative pre-trained transformer (GPT) (Radford & Narasimhan, 2018). GPT large-language models (LLMs) utilise a combination of unsupervised pre-training over massive volumes of text, as well as supervised fine-tuning and reinforcement learning on specific tasks. Due to its pre-training, LLMs generally possess broad knowledge about a wide variety of topics, and supervised fine-tuning and reinforcement learning give them the capability to perform specific tasks accurately, realistically, and in line with instructions given in natural language.

In light of the efficacy and flexibility of LLMs in synthetic data generation, I opted to utilise it for the generation of more "creative" fields in our project, such as quiz and test question generation. On the contrary, fields that have a fixed format and structure (such as test scores, student IDs) are generated using hard-coded rules.

Scalability and Adaptation

With the popularity, generalizability and rapid iteration of generative AI, it is no surprise that there is a vast selection of off-the-shelf LLMs available for use from major tech companies. OpenAI, Google, Meta, X (formerly Twitter), and Microsoft have developed their own LLMs. Most of these models are available programmatically using Application Programming Interfaces (APIs) from the respective companies, while others are available as open-source and open-weights models which can be downloaded and run on any sufficiently powerful computer or server.

OpenAI's GPT 4.1 mini (accessed via the OpenAI API) was used for synthetic data generation in this project. To avoid repetition, the capabilities of different LLMs will be discussed and compared in greater detail in Chapter 4 of this report.

All of the synthetic data generation code was written in Python and included in a Jupyter notebook. The notebook also contains code to load data into the database endpoint of choice.

Care was taken to ensure that the data generation code was designed to be as scalable and reusable as possible. For instance, generating a large number of questions sequentially using the OpenAI API was a very time-consuming task that scaled linearly with the number of questions needed. The bottleneck was primarily the inference speed of the infrastructure used to run OpenAI's models. In order to speed things up, I utilised multi-threading to send up to 10 requests in parallel, letting 10 questions be generated at the same time. This resulted in a multi-fold increase in speed and made rapid iterations and improvements much more feasible.

(Higher numbers of parallel network requests were tested, but this resulted in rate-limiting, which results in little to no improvement in speed)

Dynamic multi-threading is achieved using the `ThreadPoolExecutor()` function in Python's `concurrent.futures` library. This allows all the question generation jobs to be submitted at once, with the function dynamically managing the jobs such that if one thread finishes early, another job can be assigned to it. In contrast to batch processing where individual batches of multiple jobs are submitted sequentially, the dynamic thread assignment of `ThreadPoolExecutor` results in a significant increase in speed for jobs that have highly variable execution time, such as generative AI data generation.

Project Implementation

1. Defining the Schema Definition

I chose to use a managed SQL relational database to store and serve all our data, the details of which will be covered in Chapter 3 of this report. A relational database stores data in a tabular, row-by-column format. As such, defining a suitable data schema is essential to ensure the data generated is accurate and consistent.

Since this project is focused on learning insights and support for students specifically, I first defined the quiz_df and test_df tables to be used for synthetic data generation

The data dictionary of quiz_df is represented in Table 2.1 and test_df in Table 2.2.

Column Name	Data Type	Description	Allowed Values / Format
question_id	string	Unique identifier for each quiz question	Alphanumeric (e.g., QN14, QN47)
quiz_id	string	Identifier for the specific quiz version or session	Alphanumeric (e.g., QZ01, QZ05)
question_text	string	Full text of the quiz question presented to the student	Free-text, typically phrased as a question
subtopic	string	Specific subtopic within the module that the question is categorized under	E.g., "Data Preparation", "Predictive Analytics"
student_id	string	Unique identifier for the student who attempted the quiz	Alphanumeric (e.g., S01, S26)
module_id	string	Identifier for the academic or training module associated with the quiz	E.g., DAVA, DWBI, DSES
module_name	string	Full name of the academic or training module	E.g., "Data Science Essentials", "Data Visualization & Analytics"
attempt_number	integer	The student's attempt number for the quiz (e.g., first attempt, second attempt)	Integer between 1-3
quiz_score_percent	integer	The student's overall quiz score in percentage	Integer between 0–100
quiz_timestamp	datetime	Date and time when the quiz was submitted or recorded	ISO 8601 format: YYYY-MM-DD HH:MM:SS±TZ

quiz_duration_secs	integer	Time taken by the student to complete the quiz, in seconds	Integer ≥ 0
is_correct	boolean	Whether the student answered the specific quiz question correctly (true) or not (false)	true / false

Table 2.1: Data dictionary of quiz_df table

Column Name	Data Type	Description	Allowed Values / Format
question_id	string	Unique identifier for each question	Alphanumeric (e.g., QN61, QN14)
test_id	string	Identifier for the test session or version	Alphanumeric (e.g., TS01, TS08)
subtopic	string	Specific topic within the module that the question relates to	Pre-defined subtopics
question_text	string	Full text of the question presented to the student	Free-text, usually phrased as a question
student_id	string	Unique identifier for the student who attempted the question	E.g., S01, S44
module_id	string	Identifier for the module to which the test belongs	E.g., DAVA, LOMA, DWBI
module_name	string	Full name of the academic or training module	E.g., "Data Visualization & Analytics", "Logic & Mathematics"
test_score_percent	integer	Student's overall score on the test (in percentage)	Integer between 0–100
test_timestamp	datetime	Date and time when the test was submitted or recorded	ISO 8601 format: YYYY-MM-DD HH:MM:SS±TZ
is_correct	boolean	Whether the student answered the specific question correctly (true) or not (false)	true / false

Table 2.2: Data dictionary of test_df table

2. Using Generative AI for Synthetic Data Generation

Using the OpenAI's latest Responses API (OpenAI, 2025), I am able to send queries at scale. The primary use of generative AI based synthetic data generation in this project was to generate the quiz and test questions for the students. Generating a realistic and varied set of quiz and test questions that suit the modules and subtopics was crucial, as it would be

used to test the feasibility and reliability of the AI-enhanced solutions delivered in this project.

The instruction sent to the LLM, also known as the prompt, along with the function for code generation, is illustrated below:

```
def generate_single_question(quiz_id: str, module_name: str, qnum: int, subtopic: str, outline: list):
    prompt = (
        f"You are creating question QN{qnum} for a quiz on the topic\n"
        f"\n{{module_name}}\n"
        f"Each topic has multiple sub-topics. Ensure that the question is\n"
        f"somewhat related to this subtopic: {subtopic}.\n"
        f"Ensure that the questions are strictly in line with the learning\n"
        f"materials, a summary of which is provided to you at the end of this prompt. \n"
        f"Ensure your responses are strictly in the English language only. All\n"
        f"questions should be clear and concise, and under 25 words long.\n"
        f"Try to be as creative as possible in creating a question that is both\n"
        f"challenging and engaging. DO NOT USE COMMONLY ASKED QUESTIONS AS IT'S TOO EASY AND\n"
        f"REPETITIVE. DO NOT REPEAT QUESTIONS. \n"
        f"Outline\n{str(outline)}\n"
        f"Output EXACTLY ONE QUESTION IN THIS FORMAT:\n"
        f"<question text>\n"
    )
    # temperature = 2 gives more variation in output, but need to check if model
    # output is stable
    # else temperature = 1.5 is generally very stable
    text, status = run_model(prompt, model='mini', temperature=1.5)
    if status != "Success":
        raise RuntimeError(f"API error for {quiz_id} QN{qnum}: {text}")
    question_text = text.strip()
    return {"quiz_id": quiz_id, "question_text": question_text, "subtopic": subtopic}
```

As seen in the code block above, the module name and subtopic is given to the model, along with a concise AI-generated summary of the learning materials, such as lesson materials and slides, where available. This enables the LLM to generate high-quality, varied questions that match the learning materials given to students.

Early on in the development process, I discovered that since each question was being generated separately, the LLM had no idea which questions have already been generated. As such, it tended to generate multiple similarly-worded questions. Two plausible solutions were as follows:

- Append questions to a list as they were being generated. Pass the list as context to the model and instruct it not to generate the same question again.

- Increase the temperature of the LLM and instruct it to generate more varied results.

The former solution presented a unique technical challenge due to our use of multi-threading in Python. Several threads simultaneously reading from and appending to the question list would likely cause major issues.

After experimenting with and adjusting temperature -- the parameter controlling the randomness of the generated text -- this resulted in a set of generated questions with a greater variety and entropy.

Chapter 3: Backend-as-a-Service: SQL Database + API Infrastructure

Tools used: Supabase, PostgreSQL (within Supabase), Python

Background and Justification

When developing any software, building a reliable back-end is often the first step before starting any front-end, user-facing work. This includes setting up the database, user authentication, file storage, and server-side APIs that handles requests and transmits data between the front-end and back-end. Traditionally, the complexity of backend development necessitated significant effort and expertise to manage separate programs and servers for database, authentication, APIs and more.

In the last few years, the advent of serverless computing concepts and Backend-as-a-service (BaaS) has helped to improve the efficacy of back-end development. Serverless computing allows developers to deploy and run applications, work with databases, and interact with different services without managing the servers and physical infrastructure themselves. Due to the sheer scale of the tech firms operating serverless infrastructure, as well as the pay-as-you-go pricing structure of most serverless platforms, serverless computing enables reduced operational costs, faster development, and better scalability over self-provisioning servers and setting up backend services. (Samu, 2023).

Based on the serverless computing model, Backend-as-a-service (BaaS) platforms provide pre-built backend services like managed SQL/NoSQL databases, user authentication, serverless functions, file storage, and pre-built APIs and RESTful interfaces to interact with all those services (Cloudflare, n.d). With many benefits borrowed from serverless computing,

BaaS has enjoyed rapid industry adoption. Market research has predicted BaaS will grow from \$3 billion in 2022 to nearly \$28-29 billion in 2032 -- marking a 25% annual growth rate (Allied Market Research, 2023). As such, it is reasonable to expect that the infrastructure and applications offered by BaaS platforms will continue to expand along with market demand.

Some of the major BaaS platforms considered for use in this project are as follows:

- **Firestore:** A proprietary Google-developed platform that utilises a NoSQL database and ties in well with Google Cloud Platform (GCP) cloud services. (Roos, 2022).
- **AWS Amplify:** A highly flexible platform by Amazon Web Services (AWS) that works with many AWS services for SQL and NoSQL databases, file storage, authentication, and serverless functions. (AWS, n.d.)
- **Supabase:** An open-source platform with a PostgreSQL relational database, pre-built APIs, and many similar features as Firestore. It is generally considered to be one of the best platforms for web applications which require a relational database and prioritise quick development turnaround. (Gupta, 2024).

Among these three options, Supabase was selected for its desirable features like a PostgreSQL relational database and ease-of-use and integration. Its free tier, which includes 500MB of database space, is abundant for development and prototyping purposes. Notably, Supabase works especially well for generative AI-based applications due to PostgreSQL's support for the pgvector extension, which enables us to store vector representations of text embeddings directly in a database column. Additionally, its support for vector indexing greatly speeds up tasks like semantic search and text classification. (Richardson, 2023).

Scalability and Adaptation

During the development stage of this project, I used Supabase's Free Tier, which incurs zero operating costs. This project does not have any large-scale compute needs, as all generative AI and text embedding workloads are done via the OpenAI API. As such, Supabase's free-forever tier with its shared CPU, 500MB RAM, 500MB database storage, and shared container environment is sufficient to easily manage small-scale internal student systems with a few hundred students.

If the project increases in scale, I can upgrade to Supabase's Pro plan for \$25 USD/month. Without making any other changes, the environment will now be upgraded to 1 vCPU, 1GB RAM and 8GB database storage. This should be sufficient for larger-scale student systems with a few thousand students -- more than sufficient for Temasek Polytechnic's School of IIT.

Supabase's flexibility and scalability is a huge benefit as resources can be scaled up and down where required, without any additional development or deployment work.

Project Implementation

1. Supabase: Setting up Project and Database

As earlier mentioned, one of Supabase's key advantages is its ease-of-setup. Creating a new user account follows the same steps as creating an account on any other website. After signing up for an account, I created a self-titled project and reached Supabase's Project Overview page, as shown in Figure 3.1.

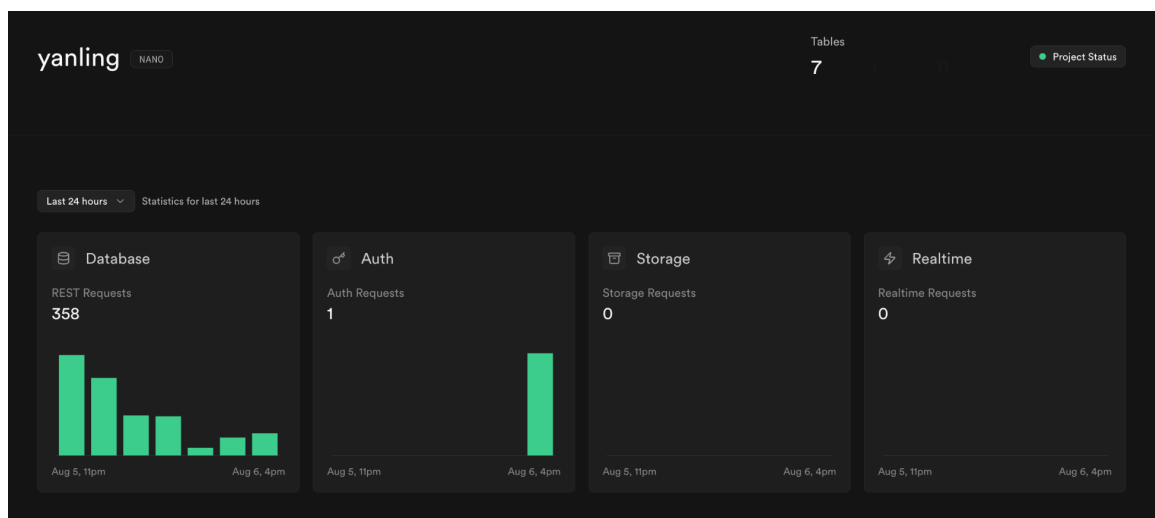


Figure 3.1: Supabase Project overview page

The first step is to set up the Supabase API to be used with my code. I copied the `SUPABASE_URL`, `SUPABASE_ANON_KEY` and `SUPABASE_SERVICE_ROLE_KEY`. These keys were added into a secure environment variable `.env` to provide greater security for sensitive information like API keys.

Next, the supabase library is needed to access Supabase's API from a Python instance. After installing the library, I created a simple environment to test basic read functionality:

```
import os
from supabase import create_client
from dotenv import load_dotenv

load_dotenv()

SUPABASE_URL = os.getenv("SUPABASE_URL")
SUPABASE_KEY = os.getenv("SUPABASE_ANON_KEY")
```

```
supabase = create_client(SUPABASE_URL, SUPABASE_KEY)

data = supabase.table("test").select("*").execute()
print(data.data)
```

2. Setting up Vector Tables to Work with a RAG Pipeline

Due to Supabase's use of PostgreSQL relational databases, it is very easy to enable support for vector tables, which are database tables that can store vector representations of text embeddings. I run the following code in the SQL editor to enable the **pgvector** extension:

```
create extension if not exists vector;
```

3. Getting around Supabase API's Limitations

Due to the limitations of Supabase's API, many regular SQL queries cannot be handled directly and natively. There's also no way to send SQL queries directly to the database engine via Supabase's API.

To mitigate these limitations, there are two main methods:

- **Using an SQL View:** Similar to a pivot table in Excel, an SQL view summarises and aggregates data based on a pre-defined SQL query. As the underlying data changes, the results of the SQL view will change accordingly. Views can help to simplify SQL queries and functions, allowing for less redundancy in writing complex expressions. SQL views are saved at the database level, not in client-side code.
- **Using an SQL Function:** (Also known as remote procedure call (RPC)). Much like a function in any programming language, SQL function lets you pass input parameters into a pre-defined SQL query, with the results being returned according to those input parameters. SQL functions are also saved at the database level, not in client-side code.

In this project, we utilise a combination of SQL views and functions to enable the efficient querying of a wide variety of data.

For example, a simple SQL function is defined as follows:

```
CREATE OR REPLACE FUNCTION all_subjects()
RETURNS TABLE(module_id TEXT)
```

```

AS $$
WITH (SELECT DISTINCT module_id
FROM quiz_df);
$$ LANGUAGE SQL STABLE;

```

It can be called using the function name and passing any input parameters (this function has none):

```

resp = supabase.rpc("all_subjects").execute()
if resp.data:
    return resp.data
else:
    return None

```

4. SQL functions

Table 3.1 presents an exhaustive list of all the functions added to our database:

Function	Input params	Output params	SQL query	Purpose
all_subjects		TABLE(module_id text)	SELECT DISTINCT module_id FROM quiz_df;	Lists all available modules
avg_subtopic_scores	p_module_id text	TABLE(student_id text, subtopic text, quiz_score numeric, test_score numeric, weighted_score numeric)	SELECT COALESCE(q.student_id, t.student_id) AS student_id, COALESCE(q.subtopic, t.subtopic) AS subtopic, q.score AS quiz_score, t.score AS test_score, ROUND((COALESCE(q.score, 0) * 0.3) + (COALESCE(t.score, 0) * 0.7), 1) AS weighted_score FROM (SELECT student_id, subtopic, MAX(quiz_score_percent) AS score FROM quiz_df WHERE module_id = p_module_id GROUP BY student_id, subtopic) q FULL JOIN (SELECT student_id, subtopic, SUM(is_correct::int) * 100.0 / COUNT(*) AS score FROM test_df WHERE module_id = p_module_id GROUP BY student_id, subtopic) t USING (student_id, subtopic) ORDER BY student_id, subtopic;	Calculates weighted scores by subtopic for a module
enumerate_modules		TABLE(module_id text)	SELECT DISTINCT module_id FROM quiz_df;	Lists all available modules
enumerate_subtopics	p_module_id text	TABLE(subtopic text)	SELECT DISTINCT subtopic FROM quiz_df AS q WHERE q.module_id = p_module_id;	Lists subtopics within a module
get_bottom_n	p_limit integer	TABLE(student_id text, overall_score numeric)	SELECT student_id, ROUND(AVG(weighted_score), 1) AS overall_score FROM student_weighted_scores_v GROUP BY student_id ORDER BY overall_score ASC LIMIT p_limit;	Returns lowest-performing students

get_correct_wrong_questions	p_student_id text, p_module_id text, p_subtopic text	TABLE(question_id text, question_text text, is_correct boolean)	<pre> WITH combined_questions AS (-- Pull from quiz_df SELECT question_id, question_text, is_correct FROM quiz_df WHERE student_id = p_student_id AND module_id = p_module_id AND subtopic = p_subtopic UNION ALL -- Pull from test_df SELECT question_id, question_text, is_correct FROM test_df WHERE student_id = p_student_id AND module_id = p_module_id AND subtopic = p_subtopic) SELECT question_id, question_text, is_correct FROM combined_questions ORDER BY is_correct; </pre>	Shows student's answers for specific subtopic
get_last_n	p_limit integer	TABLE(student_id text, overall_score numeric)	<pre> SELECT student_id, ROUND(AVG(weighted_score),1) AS overall_score FROM student_weighted_scores_v GROUP BY student_id ORDER BY overall_score ASC LIMIT p_limit; </pre>	Returns lowest-performing students
get_subtopic_scores	p_student_id text, p_module_id text	TABLE(subtopic text, avg_quiz_score numeric, avg_test_score numeric, weighted_score integer)	<pre> WITH -- for each quiz attempt, take the max per subtopic per_quiz AS (SELECT subtopic, MAX(quiz_score_percent) AS max_score FROM quiz_df WHERE student_id = p_student_id AND module_id = p_module_id GROUP BY subtopic, quiz_id), -- average those max-scores by subtopic quiz_aggs AS (SELECT subtopic, AVG(max_score) AS avg_quiz_score FROM per_quiz GROUP BY subtopic), -- average test scores by subtopic test_aggs AS (SELECT subtopic, AVG(test_score_percent) AS avg_test_score FROM test_df WHERE student_id = p_student_id AND module_id = p_module_id GROUP BY subtopic) -- combine quiz & test aggregates; calculate weighted score SELECT COALESCE(q.subtopic, t.subtopic) AS subtopic, </pre>	Individual student's performance by subtopic

			<pre> q.avg_quiz_score, t.avg_test_score, ROUND(0.3*q.avg_quiz_score + 0.7*t.avg_test_score)::INT AS weighted_score FROM quiz_aggs q FULL OUTER JOIN test_aggs t ON q.subtopic = t.subtopic ORDER BY subtopic; </pre>	
get_top_n	p_limit integer	TABLE(student_id text, overall_score numeric)	<pre> SELECT student_id, ROUND(AVG(weighted_score),1) AS overall_score FROM student_weighted_scores_v GROUP BY student_id ORDER BY overall_score DESC LIMIT p_limit; </pre>	Returns highest-performing students
get_weighted_scores	p_student_id text	TABLE(module_id text, avg_quiz_score numeric, avg_test_score numeric, weighted_score integer)	<pre> WITH quiz_summary AS (SELECT module_id, AVG(max_score) AS avg_quiz_score FROM (SELECT module_id, MAX(quiz_score_percent) AS max_score FROM quiz_df WHERE student_id = p_student_id GROUP BY module_id, quiz_id, subtopic) AS per_quiz GROUP BY module_id), test_summary AS (SELECT module_id, AVG(test_score_percent) AS avg_test_score FROM test_df WHERE student_id = p_student_id GROUP BY module_id), combined AS (SELECT COALESCE(q.module_id, t.module_id) AS module_id, q.avg_quiz_score, t.avg_test_score FROM quiz_summary q FULL OUTER JOIN test_summary t ON q.module_id = t.module_id) SELECT module_id, avg_quiz_score, avg_test_score, -- round to nearest and cast to INT ROUND(0.3 * avg_quiz_score + 0.7 * avg_test_score)::INT AS weighted_score FROM combined ORDER BY module_id; </pre>	Individual student's performance by module
match_documents	in_module_id text, query_embedding vector, match_count integer	TABLE(chunk_id text, content text, similarity double precision, module_id text, document_name text, category text)	<pre> SELECT dc.chunk_id, dc.content, 1 - (dc.embedding <=> query_embedding) AS similarity, dc.module_id, dc.document_name, dc.category FROM doc_chunks AS dc WHERE dc.module_id = in_module_id ORDER BY dc.embedding <-> query_embedding LIMIT match_count; </pre>	Finds relevant content using vector similarity

module_wrong_questions	p_module_id text	<pre> WITH combined_questions AS (SELECT question_id, question_text, student_id, subtopic FROM quiz_df WHERE module_id = p_module_id AND is_correct = false UNION ALL SELECT question_id, question_text, student_id, subtopic FROM test_df WHERE module_id = p_module_id AND is_correct = false) SELECT question_id, question_text, COUNT(DISTINCT student_id) as students_who_got_wrong, subtopic FROM combined_questions GROUP BY question_id, question_text, subtopic HAVING COUNT(DISTINCT student_id) > 25 ORDER BY students_who_got_wrong DESC, question_id; </pre>	Identifies difficult questions (>25 students wrong)
overall_summary		<pre> TABLE(student_id text, module_id text, weighted_score numeric) SELECT student_id, module_id, weighted_score FROM student_weighted_scores_v ORDER BY module_id ASC, weighted_score DESC; </pre>	All students' scores across all modules
subject_summary	p_module_id text	<pre> TABLE(student_id text, weighted_score numeric) SELECT student_id, weighted_score FROM student_weighted_scores_v WHERE module_id = p_module_id ORDER BY weighted_score DESC; </pre>	All students' scores for one module

Table 3.1: Reference table of SQL functions.

These SQL functions provide a complete set of database queries to query data from the backend, allowing for efficient representation of data on the front-end web application interface.

Chapter 4: Python Web Development with Streamlit Framework

Tools used: Python, Streamlit framework (Python library)

Background and Justification

Many tools and frameworks are available for the development of web applications. The most popular web development libraries and frameworks like React, Angular, and [Vue.js](#) primarily rely on the JavaScript programming language, along with HTML template files and CSS style

files. While these frameworks power many complex web applications available on the modern Internet, they require a good understanding of front-end web development and proficiency in the JavaScript programming language (MDN Web Docs, n.d.)

However, as a data analyst with little web development experience, writing a full web application in either JavaScript or JavaScript frameworks like React and Angular poses a significant challenge. Most data analysts are largely familiar with languages like Python and SQL as they are widely used in data querying and analysis. Due to the limited development time for this project, I have deemed it impractical to learn JavaScript and JavaScript frameworks. As such, I have chosen to develop this minimal viable product (MVP) application in a Python-based framework. This section will briefly discuss the key advantages and disadvantages of developing in a Python-based web framework over a JavaScript-based one. It will also compare the functionality and use cases of different Python-based frameworks.

Advantages of Python-Based Web Development

Python is often termed the language of the data analyst. This is not surprising as Python has a rich ecosystem of data science, machine learning, and deep learning libraries (DataCamp, 2024). Moreover, a 365 Data Science analysis of data science related job listings in 2024 showed that Python appears in more than 50% of roles, as opposed to JavaScript at 4.7% of roles (Yosifova, 2024). This indicates that the majority of data analysts and data scientists are likely to be proficient in Python, while not necessarily being proficient in JavaScript.

As such, it is much more time-efficient and resource-efficient for data analysts and data scientists to write and maintain a web application written in Python. Most Python web frameworks abstract away much of the complexity of working with specific JavaScript web frameworks and writing HTML, CSS, and JavaScript (Akkem, 2023). This enables faster prototyping and development, since the app only requires minimal boilerplate code to start running. Moreover, this reduces the tech stack and software requirements as only a Python environment with appropriate libraries is necessary to run the entire stack -- including front-end and back-end code. Consequently, this increases ease of deployment as the entire web application can be containerized and deployed as one single image (Andrews, n.d.).

Disadvantages of Python-Based Web Development

On the other hand, developing web applications in Python also comes with significant downsides. Notably, a study by Lei et al. (2014) found that Python had several times worse performance than [Node.js](#) for both I/O bound and compute intensive workloads. Its

performance stayed consistently far behind that of [Node.js](#) from low to high concurrency workloads. This is because JavaScript's V8 engine optimizes code at runtime, while Python's CPython interpreter does not make such runtime optimizations.

Additionally, JavaScript comes with a wide variety of frameworks like React and Angular, enabling the development of highly customizable and performant web applications. In contrast, most Python web development frameworks like Django, Flask, and Streamlit are designed specifically to abstract away the complexity of HTML, CSS, and JavaScript. This results in less built-in customizability for the developer.

Comparison of Python Frameworks for Web Development

There are several Python frameworks targeted for web development -- each with its own primary use case.

Streamlit is purpose-built for rendering data-centric and machine learning applications. It is used by popular data science groups such as Google X and Yelp (Streamlit, n.d.). Much of Streamlit's desirability stems from its ease-of-use, rapid development, and integration with Python's data analytics and machine learning libraries like matplotlib, pandas and TensorFlow. Its templating engine is less customizable than other options.

Flask is a lightweight Python web application framework. It is designed to enable quick and easy development for small but scalable applications (Flask, n.d.). Being a microframework, it provides a barebones and minimal design. There are ways to visualize and display data from libraries such as pandas and scikit-learn, but its integration is less robust than that of Streamlit.

Django is designed to be a complete web framework for full-stack development. It includes many built-in features such as an Object-Relational Mapper (ORM), admin and authentication system and more. It is primarily designed for larger, more feature-rich web applications.

Out of the three choices, Streamlit appears to be the best choice for its ease-of-use and ability to work well with data-heavy applications. It is notable that Stable Diffusion, one of the most popular text-to-image generators, has a web interface built in Streamlit. As such, we will be building our application using the Streamlit web framework in Python (Stable Diffusion WebUI, n.d.).

Scalability and Adaptation

Despite its primary design as a simple web framework for small-scale to medium-scale use, Streamlit does provide avenues for scalability. The application's simple project structure and minimal external dependencies (primarily just Python libraries) makes it convenient to containerize and deploy to container-based deployment platforms like Docker, AWS Fargate, and Kubernetes.

Streamlit also offers Streamlit Community Cloud, allowing for the application to be easily deployed to the cloud for small-scale use. Additionally, enterprise users can rely on Snowflake to host Streamlit applications with useful enterprise features like role-based access controls (Streamlit, n.d.).

Project Implementation

Design Considerations

Streamlit already provides a pre-built, opinionated template design and multiple UI components for laying out the web application. This includes UI elements (e.g. sidebar, columns, tabs, expandable sections, containers), UI components (e.g. text elements, media containers, charts), and interactive widgets (e.g. buttons, file uploaders). However, most of the options are rather basic and do not add any colour or style, resulting in a subjectively dull interface. As such, I opted to write some simple HTML, CSS, and JavaScript for specific design elements on the main page. For example, the subject picker and bar chart both have a similar blue-purple to hot pink gradient, and the bar chart springs open with an animation when it becomes visible to the user. Streamlit does not restrict the use of such custom design elements, and they do not noticeably impact perceived responsiveness.

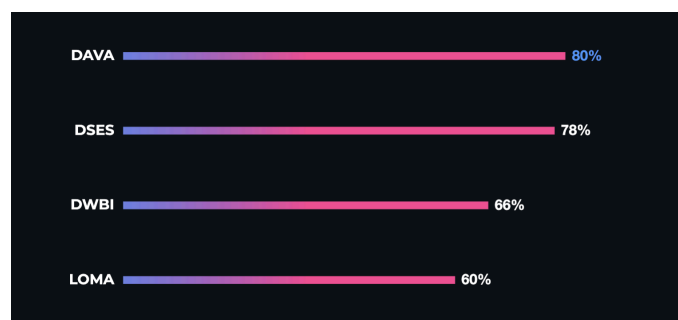


Figure 4.1: Custom bar chart written in HTML/CSS/JS

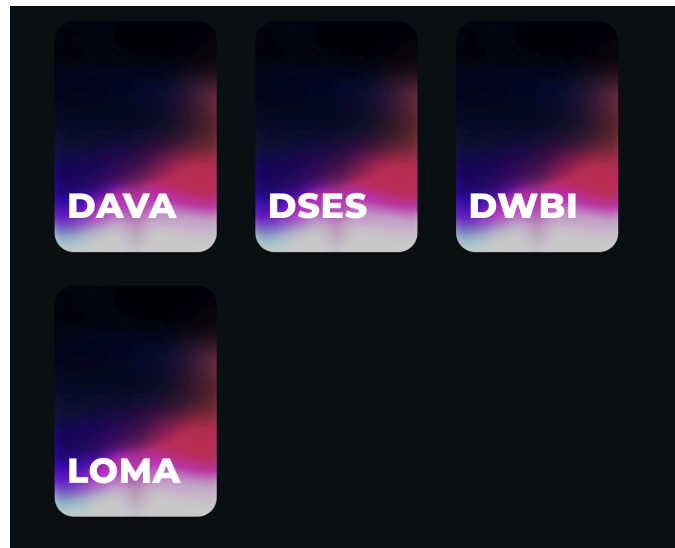


Figure 4.2: Custom subject selector written in HTML/CSS/JS. Background graphics and effects were created in graphic design software and saved as very small images to reduce computational load.

Some of Streamlit's pre-built layout components such as a sidebar and top navigation bar were completely unnecessary and posed a distraction to the user. As such, I disabled these components.

Meanwhile, there were some features that had to be built from the ground up and largely required custom HTML, CSS, and JavaScript code to be embedded. The most notable of these features is the flashcard interface. In consideration of my lack of expertise in HTML, CSS, and JavaScript, I wrote the code entirely using GPT-o3 model with about 3 to 4 prompts. (That seems rather appropriate for an AI-enhanced project). More information about the flashcard interface can be found in the Project Implementation section of Chapter 5.

Chapter 5: Generative AI features

Tools used: OpenAI models (GPT 4.1, GPT 4.1 mini, GPT 4o mini TTS, GPT 4o mini Transcribe) via OpenAI API, PostgreSQL (for RAG), OpenAI text-embedding-3-small model, FFmpeg

Background and Justification

Learning Analytics

The advancements made in big data and analytics have created new methods for educators to gain insight and understanding of their students' academic progress. In line with the demand for data-driven insights in education, the Society for Learning Analytics Research

defines Learning analytics (LA) as "the measurement, collection, analysis and reporting of data about learners and their contexts, for purposes of understanding and optimizing learning" (Siemens et al., 2011).

According to a systematic literature review of 30 articles related to learning analytics, much research in this area primarily focuses on using AI to score learners' written or spoken work and identifying high-risk students. Other studies employ dashboards, visual learning analytics (VLA) and other LA techniques. (Salas-Pilco et al., 2022). The unique combination of AI and learning analytics has provided many new opportunities for educators in the form of reduced workload due to automatic grading (Blumenstyk, 2018), as well as predictive analytics to uncover under-performing students who may be at risk of failing (Herodotou, 2021).

Moreover, modern generative AI tools allow LA tools to go beyond simple descriptive and predictive analytics. LLMs like GPT 4.1, when combined with well-constructed data analytics pipelines, can interpret and analyse large swathes of data and communicate prescriptive measures and interventions at both the classroom level and the individual student level. In addition, generative AI models' capabilities have extended beyond text generation to include generating and interpreting images, video, and speech. These are often termed multi-modal LLMs. (Cocchi et al., 2024). This report details some ways that multi-modal LLMs can be utilised to enhance targeted, personalized support for students and educators alike.

Generative AI: An Evolution In the Learning Analytics Cycle

Many students in tertiary education institutions have access to a Learning Management System (LMS), a software application to create, manage, deliver, and track their classes, exams, and training programmes (SAP, n.d.). One example is the POLITEMall learning management system, which is currently used by Singapore polytechnics and ITEs. Most LMS systems are designed as tools for students to consume course content, submit graded assessments, and view assessment grades. They may provide basic descriptive learning analytics, such as displaying a student's grades, assessment scores, and progress in consuming lesson materials. (Lambda Solutions, n.d.). However, beyond the numbers, they fail to provide clear, accessible feedback and prescriptive interventions for individual students.

To address the limitations of LMS systems, this project explores how generative AI offers a promising solution. Clow's (2012) proposed LA cycle provides a four-step analytics-driven framework for exploring and implementing generative AI models, such as multi-modal LLMs,

to improve student learning outcomes. A complementary study by Yan et al. (2024) applies Clow's LA cycle to the era of big data and generative AI, evaluating the implications that large-scale data analytics and processing and generative AI have on the field of learning analytics. Both studies served as principal references for the evaluation, design, and implementation of generative AI based features in this project. The following paragraphs elaborate on each step of the LA cycle and discuss how these modern technologies can be utilised to improve the learning experience.

Learner. Before the rise of generative AI, the role of a learner or student was distinct -- it was an individual pursuing a field of study at an educational institution or participating in some other course of study. Typically, learners take in knowledge and subject matter expertise through pre-built learning materials, in-person classes, and lectures designed by their educators.

However, some of the top multi-modal generative AI models have become highly capable in the majority of academic tasks like academic writing, problem solving in mathematics and the sciences, and answering technical questions, such as those presented in quizzes and exams (OpenAI, 2025). They can also perform research and write technical reports with academic citations (OpenAI, 2025). Consequently, learners should be increasingly expected to interact with, collaborate with, and take advantage of the strengths and capabilities of generative AI models to improve the learning process.

Data. LA is a data-centered field that utilises a wide variety of data about learners with the objective of understanding and improving their learning experience (Lang et al., 2022). This can include standardized assessment and assignment results, student engagement metrics, and textual data (such as from written assessments, assignments, and posts in university forums).

Traditionally, obtaining, collating, and loading such a large variety of data from multiple data sources was a complex and time-consuming task. Also, processing and analysing unstructured text data was a challenge for many statistics- and rule-based analytics tools. Fortunately, generative AI models natively perform natural language processing and excel at understanding, synthesizing, and generating text. This creates new opportunities for capturing, analysing and generating data. For instance, generative AI can easily format unstructured text data, understand human speech, and generate personalized educational content -- such as easy-to-read summaries, flashcards, on-the-fly personalized quizzes with explanations, and short tutorial-style videos. This fosters an engaging and interactive

learning environment where each learner can interact with and take in knowledge through a refreshing variety of written, spoken, and visual media.

Analytics. The third step of the LA cycle involves extracting meaningful insights from the data gathered from the previous step. These analytics can be broadly categorised into descriptive analytics that uses data understanding and visualization to explain historical trends and patterns, predictive analytics that combines past data with statistical and machine learning models to forecast future results, and prescriptive analytics that provides practical, actionable insights through predictive modelling and forecasting (University of Bath, 2021).

Generative AI models have shown remarkable capabilities in comprehending and processing unstructured, multi-modal data in a wide variety of formats. (IBM, 2024). This is further underscored by the wide variety of AI-powered applications that capitalise on this unique feature. A distinct example is Turbolearn, a popular student-focused web application which allows students to record lectures and upload their lesson slides, notes, and exercises in any format. Turbolearn then uses generative AI to generate summaries, create flashcards and personalized quizzes, and write answers to any of the students' queries. (Halper, 2024).

Clearly, the multi-modal nature of generative AI enables a wide variety of use cases for students. First, it provides personalized explanations suited to each student's knowledge and level of understanding. Next, it also enables dynamic, conversational systems for students to better understand and take action on their knowledge gaps. Lastly, tools such as retrieval-augmented generation can reinforce students' comprehension of learning materials provided in class (Li et al., 2025).

Interventions. The final step to complete the LA cycle is delivering timely, practical interventions to support at-risk students (Herodotou, 2021) and using multi-modal generative AI models to improve learners' knowledge and understanding. This final step "closes the gap" and makes learning analytics more impactful for the students who benefit from it most. However, this is the least explored and least developed step in existing LA studies, possibly resulting from the massive amount of manual effort required for educators to provide individual support for each of their at-risk students. This area offers the greatest opportunities for generative AI to automate the delivery of interventions to students at scale.

Such interventions can include learning materials that can be customized to the learner and by the learner. For example, generative AI models can combine data about the learner's strengths and weaknesses with prescriptive analytics to recommend specific types of

exercises, quiz questions, and topics for the student's needs. Students can also decide what format (e.g. text, audio, video) they would like to receive LA interventions in, utilising the ability of generative AI models to generate multi-modal content. A related case study can be seen in the FLORA project, which utilises multi-modal learning analytics to combine learners' prior knowledge and self-regulated learning (SRL) processes to generate adaptive, personalized scaffolding, improving the efficacy of SRL for students. This further underscores the importance and effectiveness of personalized interventions in improving the learning experience for students (Li et al., 2023).

In summary, the literature depicts how generative AI and multi-modal large language models can mitigate many of the gaps in learning analytics, providing a personalized, interactive, and data-backed learning experience for each student. These insights align directly with the Problems Identified section of this report. The Project Implementation section of this report will detail how these insights influenced the design choices made in this project.

Advanced Generative AI Techniques

Most generative AI features are relatively simple to use and program. This is largely due to the fact that most generative AI interactions occur in natural language. Consequently, users and developers alike can simply give the generative AI model instructions in plain English and receive responses in natural language. However, some advanced generative AI techniques further unlock the potential of generative AI by enabling new use cases outside of typical question-and-answer scenarios.

Retrieval Augmented Generation (RAG). While LLMs possess broad world knowledge about a plethora of topics, its knowledge is limited to publicly available, historical information that was used during its pre-training. Most LLMs also have limited context sizes and tend to experience signal degradation at larger context sizes. Broadly, this results in unique challenges when using massive amounts of external knowledge sources such as learning materials and classroom slides to improve an LLM's domain expertise (Wu et al., 2025). Firstly, these sources of information are not publicly available, so they are unlikely to be in the LLM's training data. Secondly, they would not fit into the LLM's memory, so it is unlikely to be effective at instructing the LLM on its own without further techniques. Thirdly, due to the "lost in the middle" phenomenon caused when transformer-based LLMs spreads its attention across a large number of tokens, most of the data gets degraded or lost when a large amount of text is given to an LLM at inference time (Qin et al., 2022).

RAG mitigates most of these problems. Large volumes of textual documents are first encoded using a text-embedding model such as OpenAI's text-embedding-3-small model. This encoding converts chunks of a few paragraphs each into a vector, each containing a large array of numbers. Next, these vector embeddings are stored in a vector database. When queries are made, a similarity search is made by comparing the vector embeddings with that of the vector embedding of the query. Then, the top k chunks of text which are most similar to the query are retrieved along with their metadata. This text is used to augment the prompt, giving the LLM richer contextual information (Gao et al, 2023). Figure 5.1 depicts a basic retrieval augmented generation (RAG) system.

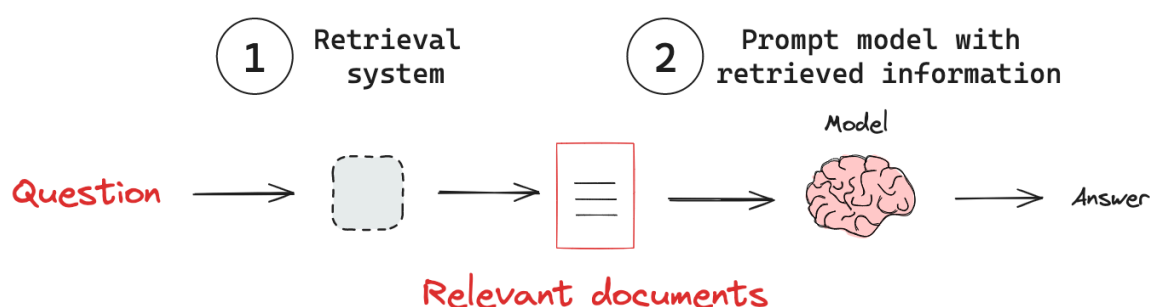


Figure 5.1: Basic retrieval augmented generation (RAG) system (LangChain, n.d.)

There are many advancements and techniques to improve the basic retrieval augmented generation system detailed above. Some of them include summary embedding, windowing, metadata filtering, and fine-tuning, as detailed in this informative LangChain guide (n.d.). Due to the limited development time for this project, only the basic RAG system was implemented to retrieve learning materials from the database for specific AI features detailed in the Project Implementation section. However, the modular structure of this project's architecture allows for easy integration of these advanced RAG features in the future.

Scalability and Adaptation

Generative AI is a rapidly shifting field with new models introduced every few weeks. New models typically promise increased performance, lower cost, and more functionality.

Since this project does not require deep reasoning or problem solving capability, we have opted not to use reasoning models such as GPT o3, GPT o4-mini, and Gemini 2.5 Pro. While reasoning models excel at solving highly complex questions in math and coding, these capabilities typically come at the cost of very long inference time (resulting in long waiting times for the user) and exponentially higher cost. These reasoning models also provide little to no benefits over non-reasoning models for primarily knowledge-based tasks (Raschka, 2025).

I chose to use GPT 4.1 mini for its relatively low price, good subjective performance during development and testing, and due to my relative familiarity with navigating the OpenAI API Platform. Other models, such as Google's Gemini 2.5 Flash are worth testing for future iterations of the project.

However, I do recognise that this model would likely be outclassed in several months' time when better performing and more affordable models are released. As such, I've ensured that all API calls to the OpenAI API go to a single function, allowing for the project to be easily migrated to new models or models from a different company by merely modifying a few lines of code. This allows for the entire project to be adapted to a new model or a different AI provider's API with ease.

Project Implementation

Based on the key insights gathered from the literature review, I have carefully designed the generative AI features in the AI-Enhanced Student Support Portal to meet the needs of educators and students alike. In no particular order of importance, the key generative AI features include:

For Students

- A customized, motivational **welcome message**
- A concise **per-module summary** containing both descriptive and prescriptive analytics
- Detailed **per-subtopic exploration** of incorrectly answered questions. Students can get a personalized AI-generated answer.
- **AI-powered revision flashcards** give students a fun, interactive way to improve their understanding of weak topics. These short-answer, relatively simple questions are generated by AI, based on past questions they have answered wrongly.
- An **all-new AI-powered interactive learning environment**. This environment takes full advantage of the multi-modality that modern generative AI models provide. It uses lesson slides, assessment results, correct/wrong questions to create interactive and personalized AI-generated instructional videos with automatic captions and voiceover based on the selected subtopic. These short videos are purpose-built to resemble popular social media content to engage young students. Students can also interact with the AI to ask any relevant questions that they have.

For Teachers

- An **in-depth analysis and automated email reporting** of the strengths and weaknesses of the lowest-performing students, at a subtopic level.
- **Real-time AI summary and data visualization** of class performance metrics as a whole.
- **Real-time AI summary** of common mistakes and weak topics as a whole, at a subtopic level.

Below is a summary of the methodology used to bring these features to life:

AI Feature 1: Motivational Welcome Message

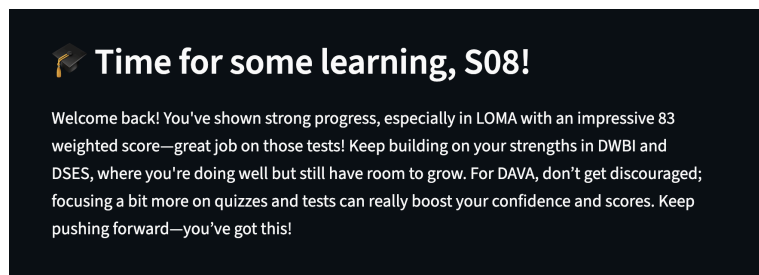


Figure 5.2: Motivational welcome message

The student's overall scores for each module is extracted and sent to the AI model. A personalized welcome message is then written based on those figures. Specific prompting was used to always maintain the positive and motivational tone of the message. The response is also cached to avoid having to reload the message when navigating to and from the home page.

AI Feature 2: Per-Module Summary

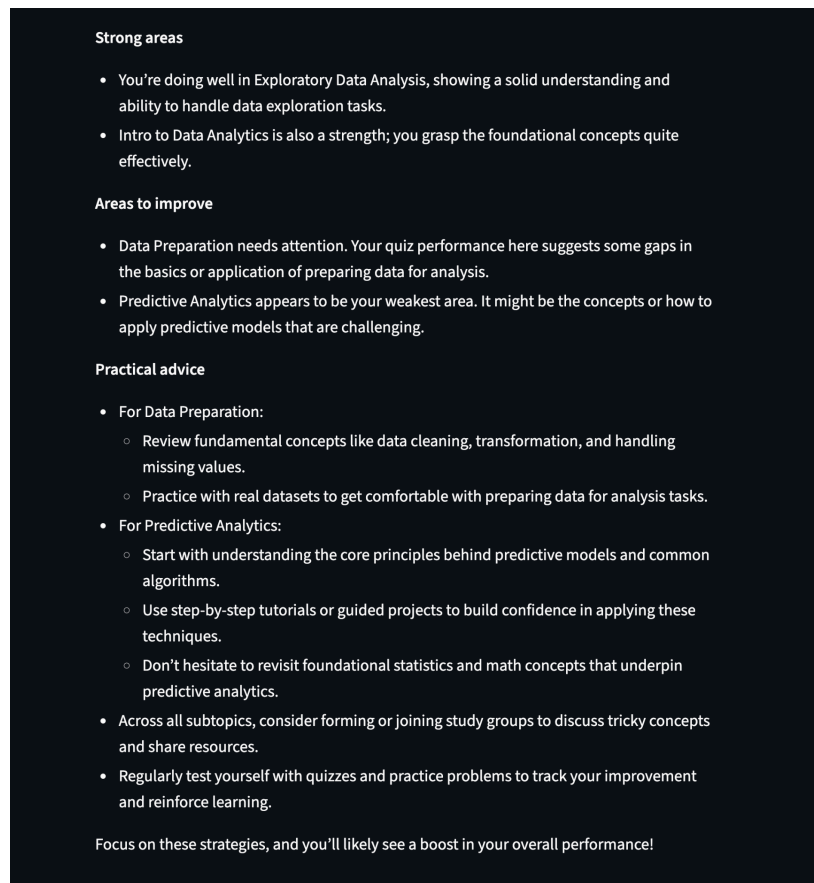


Figure 5.3: Per-module summary

In the individual module view, the student's scores are obtained by subtopic and the data is submitted to the AI model for analysis. Specific prompting was used to always maintain the structure of the result -- bulleted points under Strong areas, Areas to Improve, and Practical advice. Practical advice would focus on the subtopics that students are weak at.

AI Feature 3: Detailed Per-Subtopic Exploration of Incorrectly Answered Questions.

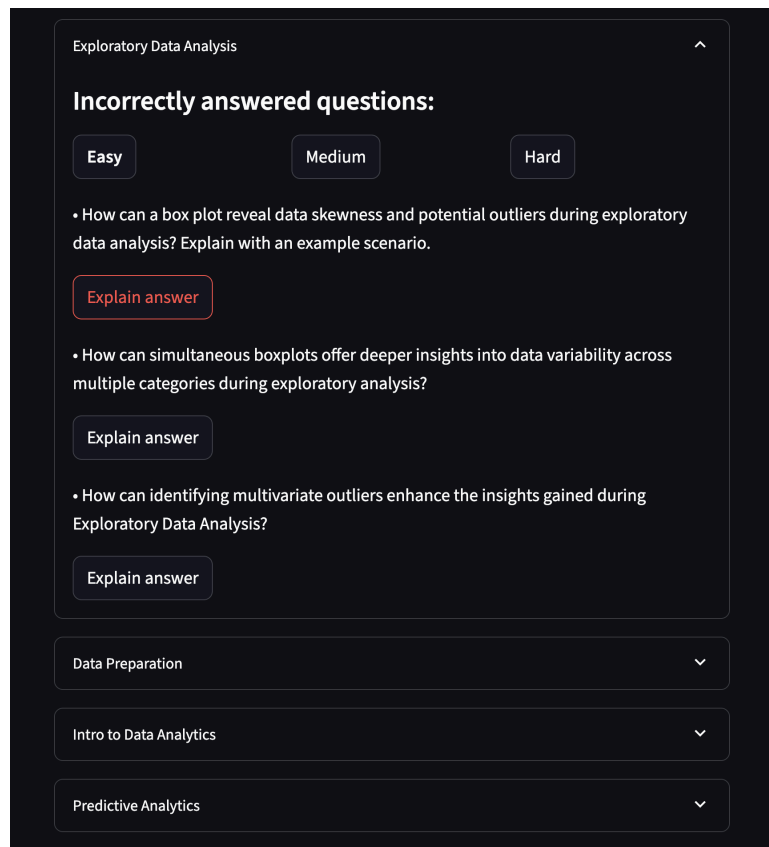


Figure 5.4: Per-module summary

In the Individual module view, students can explore the questions they have gotten wrong during quizzes and tests. These questions are automatically grouped into Easy, Medium and Hard difficulties, enabling students to customize their desired difficulty level.

When the Explain answer button is clicked, a retrieval augmented generation (RAG) pipeline is utilised to find the top 5 learning materials that are similar to the question asked. If the question is sufficiently similar to one or more relevant pieces of learning materials, the AI model will prioritise using their content to answer the question. If the question does not meet the similarity threshold (it is dissimilar from most pieces of learning materials), the AI model will only use its own knowledge base to answer the question.

For this feature, the AI model is specifically prompted to answer in a detailed yet still easy-to-understand way, helping students to learn to understanding and take in new concepts.

AI Feature 4: AI-powered Revision Flashcards

Flashcards are a powerful way for students to learn key concepts in a fun and engaging way, and this method of revision has been shown to aid students' memorization and active recall (Pass, 2024).

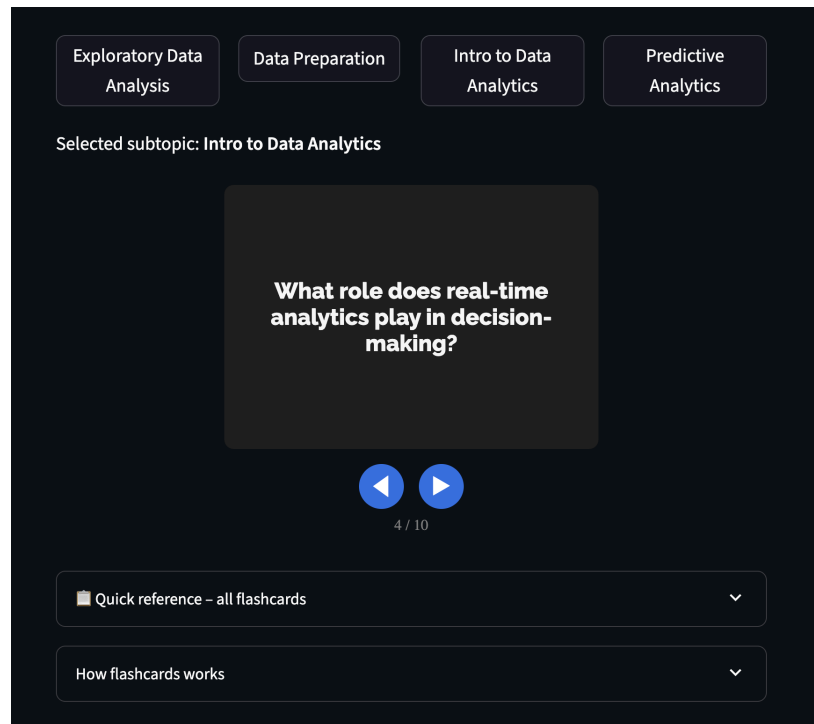


Figure 5.5: AI-powered revision flashcards

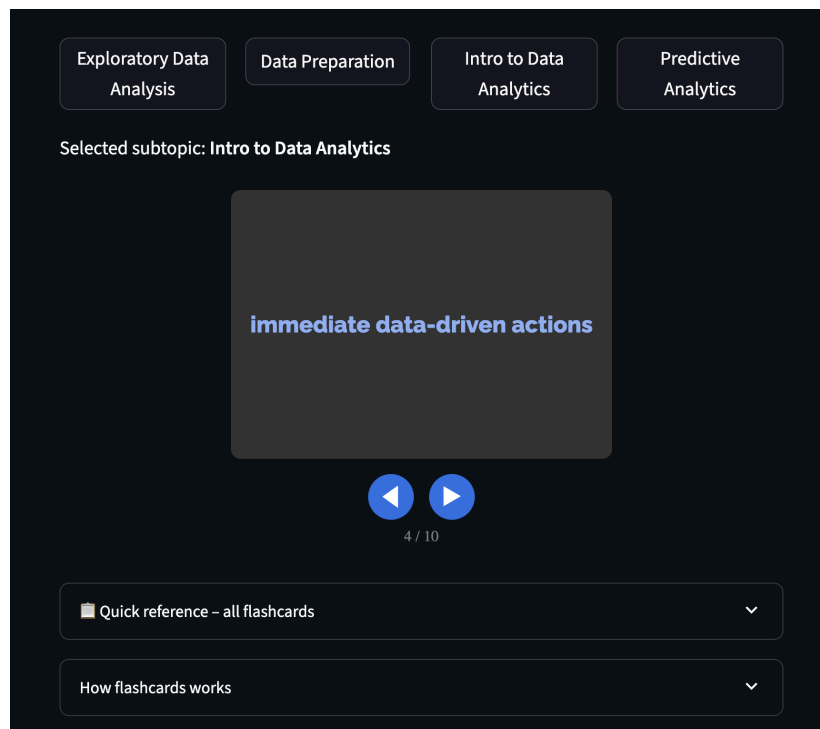


Figure 5.6: AI-powered revision flashcards: when clicked

In this feature, the student's wrong questions are fed into the AI model, together with specific instructions on the types of questions (and their corresponding answers) to generate.

Students can use the left/right arrow keys to navigate between questions, and clicking on the

flashcard flips it over to reveal the correct answer. The "card flip" animation is aesthetically pleasing and adds another layer of interactivity to the user experience.

There is also an expandable menu allowing students to see a list of all of the flashcard questions along with their answers.

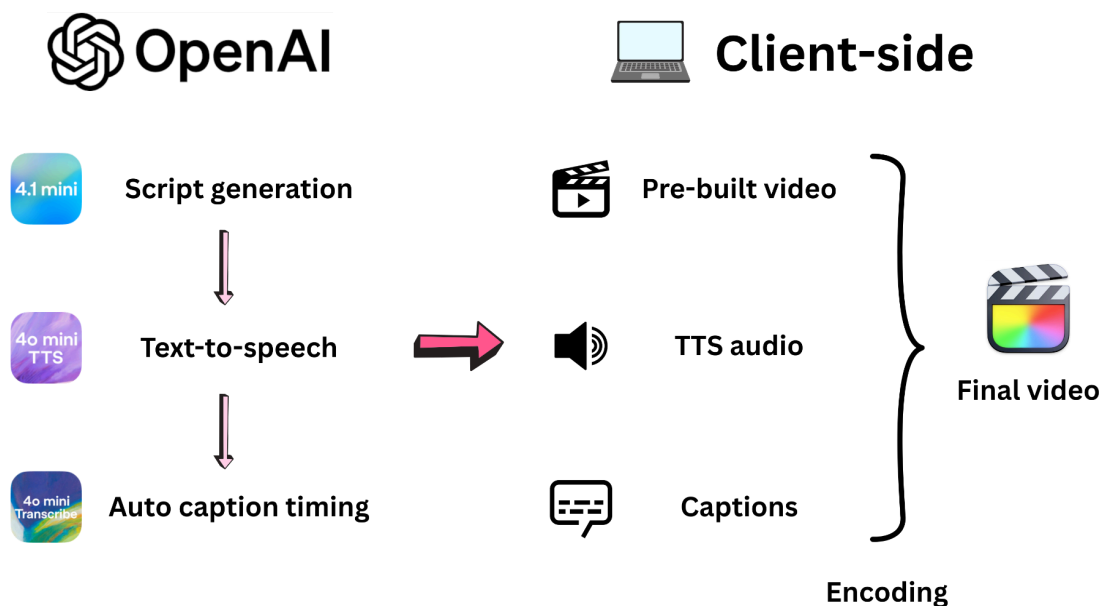
For this feature, the AI model was specifically prompted to give short-answer questions that reinforce foundational concepts. This is in line with the atomic principle of effective flashcard design, which recommends only including a single idea, fact, or question per card (Khan, n.d.).

AI Feature 5: AI-powered Interactive Learning Environment

The Interactive Learning mode is a breakthrough feature that was only made possible with the massive advancements in multi-modal generative AI technology and infrastructure over the last two years. It utilises a combination of text, audio, and video to engage students and enable them to have meaningful conversations about topics that they're interested in.

Of course, using emerging technology comes with unique challenges, particularly high costs and unproven reliability. As earlier discussed in the Background and Justification section, generative video is still very costly. As such, it is only economically viable if the generated video has substantial economic value, such as in video advertisements. While the technology for generating AI videos at scale is present, its high cost makes it unfeasible for education and student support purposes (Kapoor & Kumar, 2025).

On the other hand, audio generation (also known as text-to-speech) and transcription (also known as speech-to-text) models are relatively affordable to use via the OpenAI API, with most audio-related models costing slightly more than their text-based counterparts (OpenAI Platform, n.d.)



5.7: Illustrative step-by-step diagram depicting the workflow of generating videos

Figure

As illustrated in Figure 5.6, the workflow of generating videos involves a sequential, multi-step process as follows:

- The user first selects from three types of videos available -- learning new content, revision, and explaining past mistakes.
- When the subtopic is selected by the user, relevant data such as **assessment performance, analysed strengths and weaknesses, lesson materials, and exact correct/wrong questions** are extracted.
- The extracted data along with specific, relevant prompts are passed to GPT 4.1 mini, which generates a **video script**. Specific instructions are used to ensure the video adheres to a pre-defined structure and the learning materials provided.
- The video script is passed to GPT 4o mini text-to-speech (TTS) along with relevant parameters such as voice type and speed. **An audio file** with an AI-generated orator is created and downloaded to the application client.
- The audio file is passed to GPT 4o mini Transcribe. This is necessary to obtain precise timings for the burned-in captions. The **caption timings** are downloaded to the application client.
- At the application client level, an open-source video encoding library (FFmpeg) is used to combine video, audio, and captions in a process known as **video encoding**. Where available, hardware video encoding is used on Windows and Mac computers to greatly speed up the process to around 5 seconds or less on modern systems (FFmpeg, n.d.)
- The end result is a **final video** shown to the user, along with a concise textual summary of the video's main points.

While the actual workflow is relatively complex, the user experience is simple. Users only have two selections to make: the type of videos (learning new content, revision, explaining mistakes) and the subtopic. This enables a clean, frictionless environment for students to interact with and learn from AI, as seen in Figure 5.7.

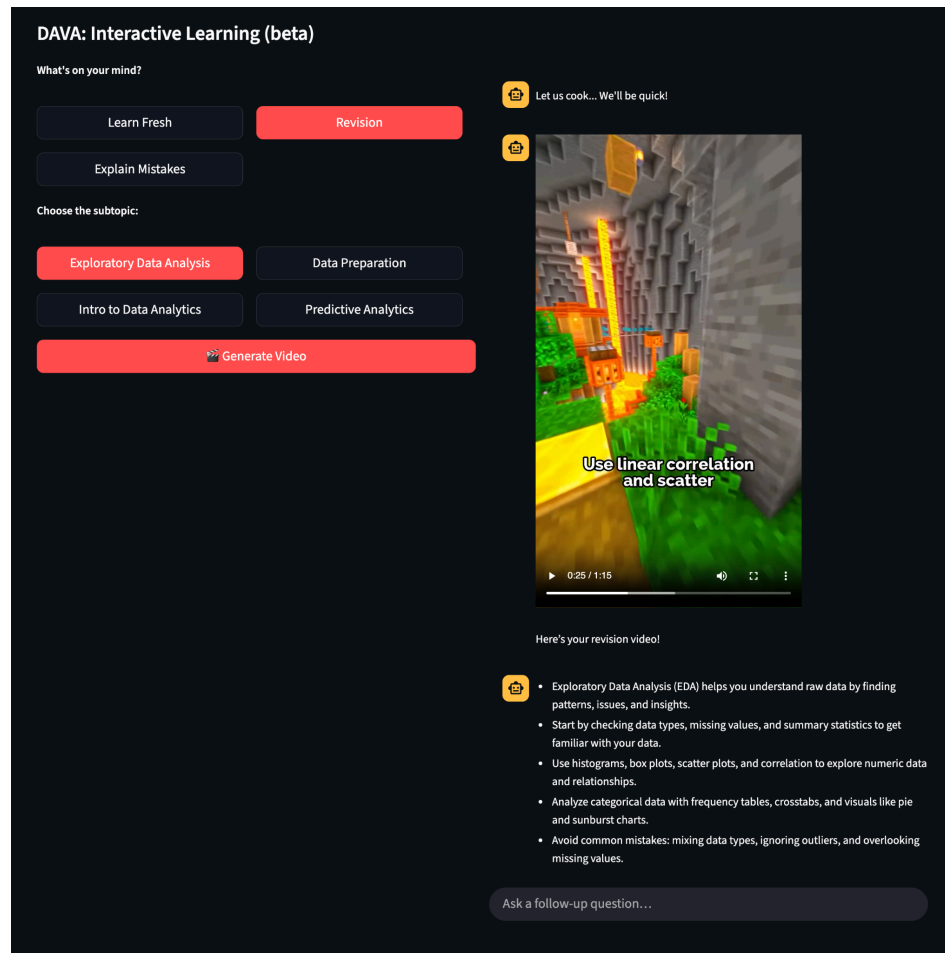


Figure 5.8: Interactive Learning interface

Besides generating videos, students can type responses to the AI model using the input box at the bottom. All chats have in-context memory -- all prompts submitted will be augmented with the current session's chats and all the information passed to the model, such as the student's extracted data. This allows the model to provide contextualised, personal responses, further enabling a unique, personalized experience for interactive learning.

AI Feature 6: In-depth Descriptive and Prescriptive Analysis of Low-Performing Students + Automated Email Reporting

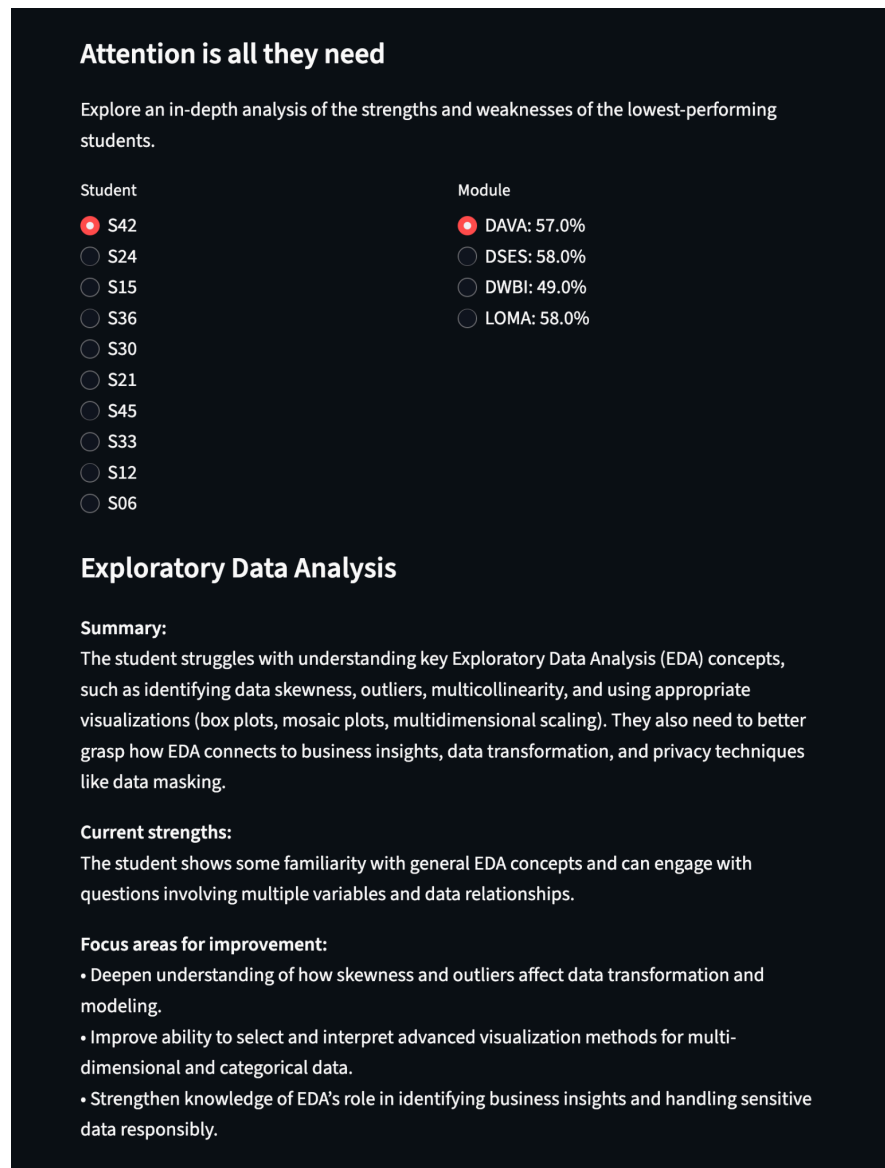


Figure 5.9: In-depth analysis of low-performing students

The 10 lowest performing students are filtered out using their assessment results. Their proficiency in different subtopics is analysed based on extracted data. Specific prompting is used to ensure that the AI model consistently provides the same structure of descriptive and prescriptive analyses, ensuring fair and unbiased advice is given to educators.

As shown in Figure 5.8, the Focus areas for improvement section provides a good starting point for educators to prescribe specific interventions for students, whether through personal coaching or providing additional online materials. It outlines specific topics and weaknesses that should be addressed based on data-driven analysis, eschewing the need for educators to perform complex analysis manually.

Summaries about the lowest performing students can be automatically sent to educators by submitting their email into the system.

AI Feature 7: Real-Time AI Summary and Data Visualization of Class Performance Metrics

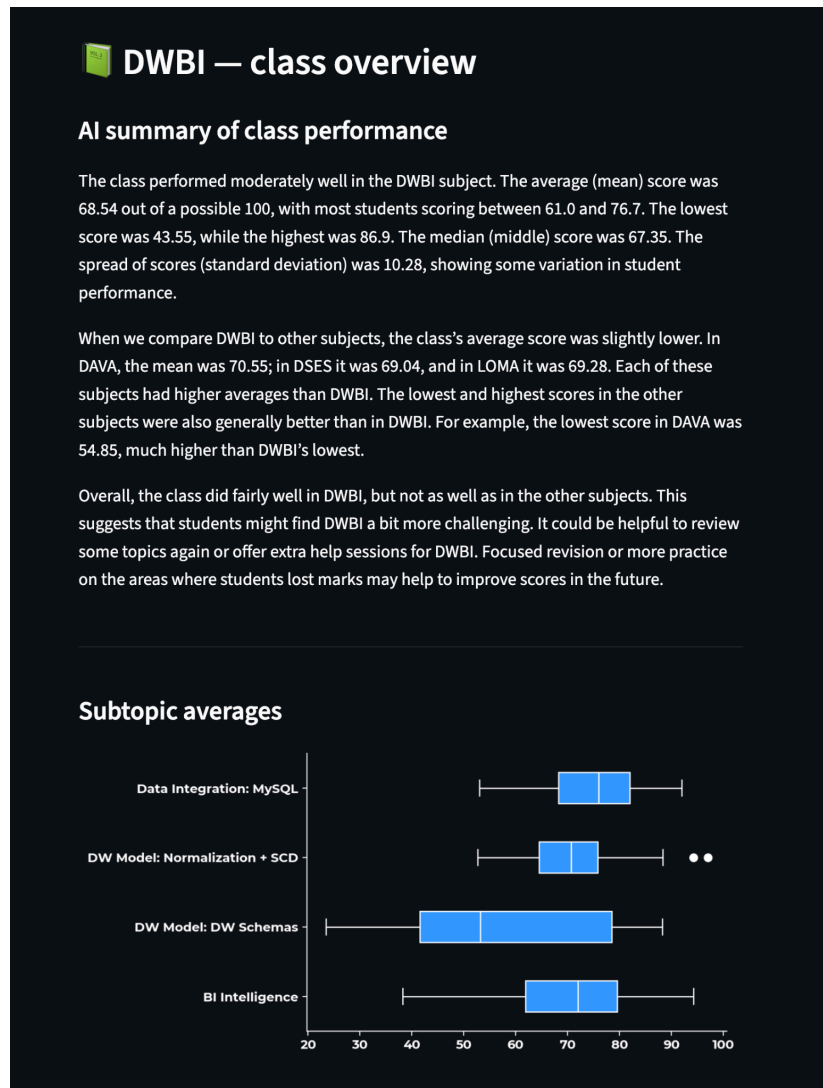


Figure 5.10: AI summary and data visualization by module

In the individual module view, descriptive statistics of the overall scores are obtained by subtopic and the data is submitted to the AI model for analysis. The AI puts the numbers into words -- providing clear, actionable insights for the educator to work with.

AI Feature 8: Real-time AI Summary of Common Mistakes

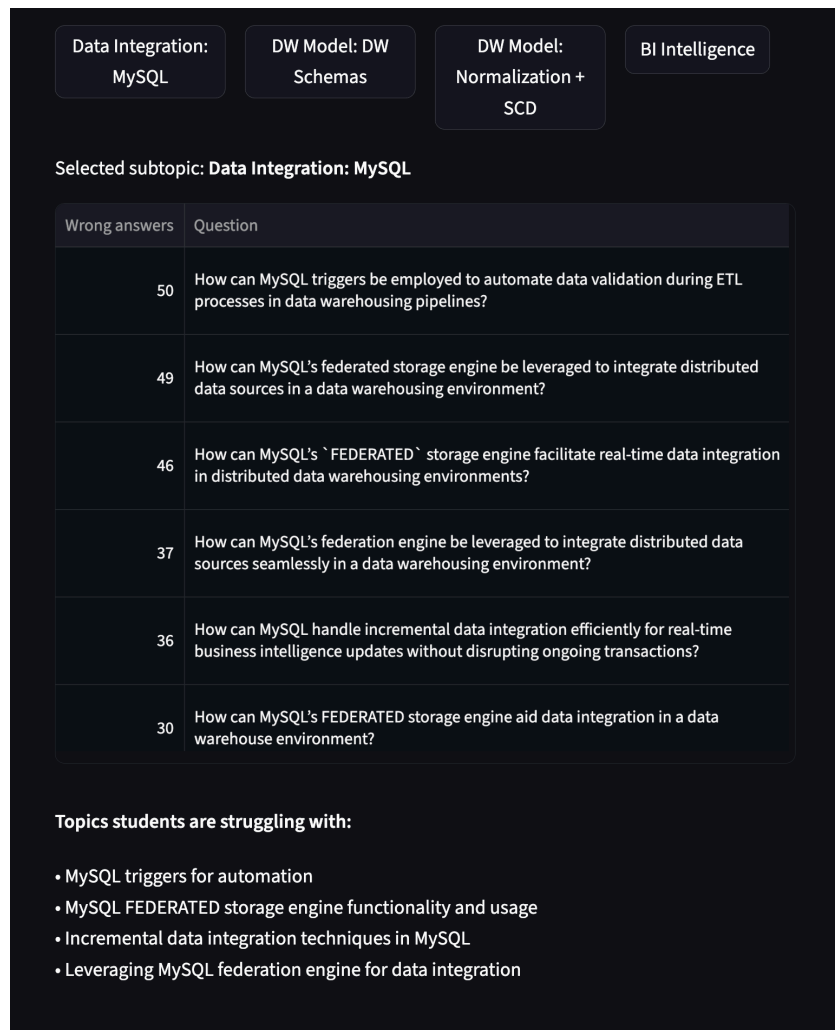


Figure 5.11: AI summary of common mistakes

Among all the assessment questions posed to students in quizzes and tests, the ones where more than half the class answered wrongly are highlighted in this table. Questions with a high rate of incorrect answers indicate a lack of understanding in the relevant topics. The AI model is used to summarise the weak topics in the class, providing educators with an efficient way to plan the next lesson's revision topics or send out relevant learning materials to their students.

Chapter 6: Data Visualization and Predictive Modelling

Tools used: Tableau (Dashboarding), Python (ML model)

Background and Justification

In recent years, data-driven approaches in education have gained popularity, particularly with the integration of machine learning (ML) models and interactive data visualization tools. These technologies allow educators to monitor student performance and progress and provide assistance more effectively.

Python is widely recognised as the leading programming language for machine learning due to its extensive libraries, simplicity and scalability. Libraries such as scikit-learn, pandas, and NumPy allow for efficient data manipulation, model training and evaluation. Specifically, the k-means clustering algorithm is used in this project to group students based on their performance. These clusters help identify learning profiles, such as high performers and students who have the same area(s) of weaknesses, allowing lecturers to be able to tailor studying materials to bridge the studying gap (Brempong, 2021).

Tableau, a widely adopted business intelligence tool, complements this process by transforming raw data and model outputs into interactive dashboards. Tableau's visual engine allows users to build charts, graphs and filters with minimal coding, making it easy to navigate and use for both technical and non-technical users (Tableau Software, n.d.). Its' support for dynamic filters allows for creation of user-specific dashboards. This feature will be useful for lecturers to control the parameters to view different learning performance and for students to track their personal performance.

Scalability and Adaptability

The K-Means Clustering model in this project is designed with scalability and adaptability in mind. With the preprocessing step of standard scaling, the model can effectively handle datasets of varying feature scales, ensuring consistent performance even if the dataset or the number of features gets bigger. The use of automated Elbow method via the Yellowbrick library allows for automated selection of optimal number of clusters, making the model adaptable to different data distributions without the need for manual tuning. The structure of the code also allows easy adjustment of key parameters, such as range for number of clusters (kmin, kmax) and random seed settings, simplifying experimentation process and future improvements. This flexibility ensures that with the availability of new student data or

performance metrics, the model can be easily retrained and scaled without requiring major changes to the code.

The tableau worksheets in this project are directly connected to Supabase, instead of the CSV files, which significantly enhances scalability and adaptability. By using Supabase as the data source, the dashboard can dynamically query real-time data, enabling the system to efficiently handle growing data and increasing number of users without affecting performance. This live connection also allows the updates to the data to be instantly reflected in the visualizations, eliminating the need for manual data refreshes. Additionally, Supabase's flexible database structure allows for easy modification and expansion of datasets, such as adding of new performance metrics/learning modules. As a result, Tableau dashboards can be changed quickly to accommodate evolving requirements with minimal reconfiguration, supporting ongoing development and improvement of the analytics platform.

Project Implementation

Machine Learning Model

1. Transformation of Data

Before training the K-Means model, data transformation was done. First, I aggregated the quiz_df and test_df by student_id, module_id and subtopic. This allows each row of data to be unique in terms of module_id and subtopic, instead of having multiple rows of record for one student for one module_id and subtopic. This allows the data to be in the correct format for model training, without multiple records of data for one module_id and subtopic, which can introduce noise during the clustering process. Below is the code used to aggregate both of the datasets.

```
quiz_df_agg_subtopic = quiz_df.groupby(['student_id', 'module_id',  
    'subtopic']).agg({  
    'quiz_id': 'unique',  
    'quiz_score_percent': 'max'  
}).reset_index()  
  
quiz_df_agg_subtopic.sort_values(by=['student_id', 'module_id',  
    'subtopic'], ascending=True)
```

For the test_df, the percentage score for each student by each subtopic was calculated. It is done using the 2 blocks of code below:

```
test_df_agg_subtopic = test_df.groupby(['student_id', 'module_id',  
    'subtopic']).agg({'is_correct': 'sum',
```

```

        'question_id': 'nunique'}).reset_index()

test_df_agg_subtopic.rename(columns={
    'is_correct': 'is_correct_by_subtopic',
    'question_id': 'question_count_by_subtopic'}, inplace=True)

test_df_agg_subtopic.sort_values(by=['student_id', 'module_id',
    'subtopic'], ascending=True)

```

```

test_df_agg_module = test_df.groupby(['student_id', 'module_id']).agg(
    {'is_correct': 'sum',
    'question_id': 'count'}).reset_index()

test_df_agg_module['test_percentage_correct'] =
    (test_df_agg_module['is_correct'] / test_df_agg_module['question_id'] *
    100).astype(int)

test_df_agg_module.drop(columns=['is_correct', 'question_id'],
    inplace=True)

```

This transformation allows performance for tests to be measured as a proportion rather than an absolute score, allowing for fairer comparisons, especially across different subtopics which may have different numbers of questions.

The percentage score for quiz and test datasets for each student is merged together, this allows for both assessment types to be compared together in the clustering model. This overall view is able to reflect each student's overall learning performance and not just their results by each module.

	student_id	module_id	quiz_score_percent	test_percentage_correct
0	S01	DAVA	77.5	88
1	S01	DSES	75.7	86
2	S01	DWBI	70.0	62
3	S01	LOMA	76.0	64
4	S02	DAVA	62.5	60
5	S02	DSES	70.0	60
6	S02	DWBI	80.0	74

Figure 6.1: Outcome after merging of percentage score

Lastly, to organize the merged dataset into a suitable structure for machine learning, I created a new column which combines module_id with the metric name (test/quiz). This was followed by pivoting of the table so that each student_id is a single row of data, and each unique module-metric pair becomes a separate column. This pivoting transforms the dataset

into a wide format, allowing each student's performance across different modules to be stored as individual features in the model. This format gives the model a comprehensive view of how each student performs across multiple modules and assessments in a single, unified format. The dataset was also scaled before being used in the K-Means model. This step is crucial as K-Means clustering is distance-based and sensitive to varying scales. Without normalization, features with larger numerical ranges can disproportionately influence the clustering output. Scaling ensures that each metric contributes equally to the formation of clusters, resulting in more balanced and meaningful groupings.

	DAVA_quiz	DAVA_test	DSES_quiz	DSES_test	DWBI_quiz	DWBI_test	LOMA_quiz	LOMA_test
0	0.595790	1.336313	0.866393	1.299724	0.192108	-0.503111	0.750539	-0.483453
1	-0.857357	-0.763393	-0.009217	-0.631294	1.085633	0.384732	-0.007581	1.395692
2	0.595790	-0.013498	0.866393	-0.854104	0.415489	-0.947032	0.561009	-1.081363
3	0.837981	1.636271	0.436269	1.596804	-0.478036	-0.503111	0.750539	-0.312621
4	1.080172	-0.013498	-0.009217	-0.185675	-2.041705	1.420547	-0.576171	0.370704

Figure 6.2: Outcome after pivoting of dataset

2. Training of K-Means Clustering Model

Using the elbow plot, 4 was picked as the best k number as it has the highest drop in the SSE number. Using that number, the K-Means Model was plotted and the cluster labels were assigned to each row of data. Using the assigned clusters of data, a pairplot was plotted to see the relationship between each cluster to the data.

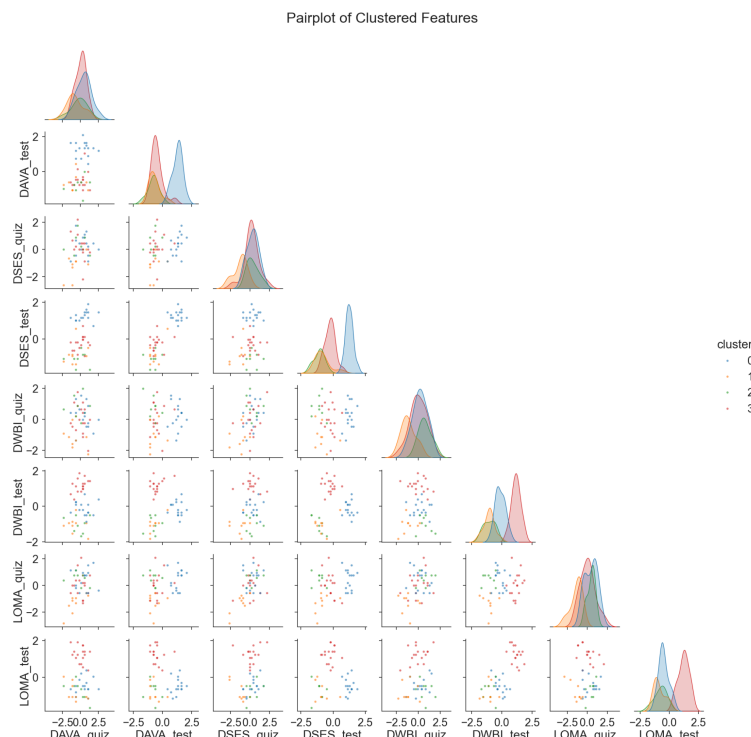


Figure 6.3: Pairplot of clustered dataset

From the pairplot above, we can see that:

- **Cluster 0:** Perform well for DAVA and DSES test, but has a variety of results for DWBI, LOMA, DAVA and DSES quiz
- **Cluster 1:** Generally don't perform well for both test and quiz across all modules
- **Cluster 2:** Don't perform well for test but performs better than Cluster 1 for quiz across all modules
- **Cluster 3:** Perform well for DWBI and LOMA test, but has a variety of results for DAVA, DSES, DWBI and LOMA quiz

With the results obtained, we can see that students in Clusters 0 and 3 have modules that they perform well and don't perform well in. Lecturers just need to provide more assistance to students in the 2 respective modules they are not performing well in. Whereas for students in Cluster 1 and 2 requires more attention in bridging the gap across all modules. Lecturers can provide more learning materials/consultations for students for all modules in these 2 clusters to bridge the learning gaps. It is also interesting to note that students in Cluster 2 perform well for quizzes but not during tests, this highlights the need for lecturers to pay extra attention to these students and enquire if they are truly understanding the concepts.

Tableau Dashboard

To get the data to plot the charts, connection to Supabase is configured.

1. Configuration in Supabase

In Supabase:

- Click **Connect** at top of screen, then copy the URL in the Session pooler. This already contains the server host, port, database name, username and password needed for configuration for connection of data on Tableau.

```
postgres://postgres.wlkslwswsqcewvubwvms:[YOUR-PASSWORD]@aws-0-ap-south-east-1.pooler.supabase.com:5432/postgres
```

2. Configuration in Tableau

In Tableau:

- Click '**Other Databases (JDBC)**' to start establishing connection to the database. Enter the URL above into the URL box in Tableau and change the Dialect to PostgreSQL (if necessary).
- Click '**Sign In**' to establish the connection with the database
- Change the Schema to '**public**' and all the datasets that are stored in the Supabase database will be shown as shown below

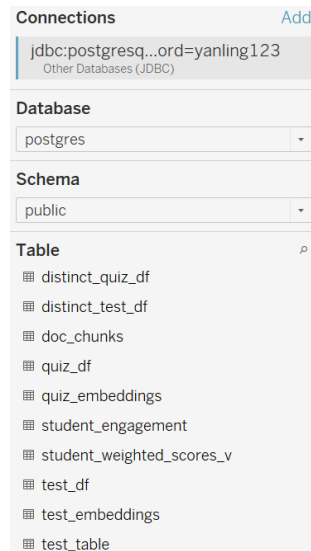


Figure 6.4: Screenshot of datasets obtained after establishing connection with Supabase on Tableau

- These datasets can all be used to create the charts for the students and lecturers dashboards.

3. Design and Interpretation of Dashboards

Student Dashboard



Figure 6.5: View of Student Dashboard

The student's dashboard is designed in a manner where the students are not able to view the actual mark they obtained for each module for both quizzes and tests separately but instead are only able to view the questions percentage breakdown results. This allows students to still be able to identify their areas of weaknesses without the need of releasing

their actual marks. The filters at the side for the subtopics allow students to easily choose the subtopic that they want to view their performance for.

For the dashboard above, it is filtered for student_id S01, we can see that in terms of quiz, the student answered almost half of the questions incorrectly for DAVA and DWBI. It is a bit better for DSES and LOMA with around 30% of the questions answered incorrectly. For the tests, the student does better for DAVA and DSES, with only around 12-14% of the questions being incorrectly answered whereas for DWBI and LOMA, the student does not do as well, with around 36-38% of the questions incorrectly answered. Cooperating with the insights obtained from the K-Means clustering model, we are able to derive that this student belongs to cluster 0, with better performance in DAVA and DSES test and a variety of performance in the other modules test and all quizzes.

The student is able to see the improvement in results from the quiz to the test for DAVA and DSES, with this information, the student is able to know that the learning method or change in learning style for DAVA and DSES is working and they should apply those learning methods to the other modules in order to improve their overall results.

The student is also able to filter the graphs by the number of times they attempted the quizzes. This allows students to understand whether their performance for each module/subtopic had improvements as they attempted the quizzes more. Students are then able to know if they have actually understood the concepts taught for that module/subtopic. If their performance improved as they attempted the quizzes more, it shows that reviewing the topics and reattempting the quizzes does help them in understanding the concepts better. If their performance does not improve as they attempt the quizzes more, it shows that quizzes are not an effective way of learning for them and they should find other alternatives to understand the concepts first and then reattempting the quiz instead of just doing the quizzes blindly. The student can also view the percentage of materials they had viewed for the respective modules, which allows them to know if a poor performance in a certain module is related to them viewing too less of the lesson materials or they just simply couldn't understand the concepts taught, which is a signal to the student that they should be approaching their lecturers to clarify any doubts and bridge the learning gaps in order to perform better.

There are other charts that are implemented in the Streamlit web application which brings across similar points as some of the points in the dashboard above but it is more to complement the GenAI portion of the page.

Lecturer Dashboard

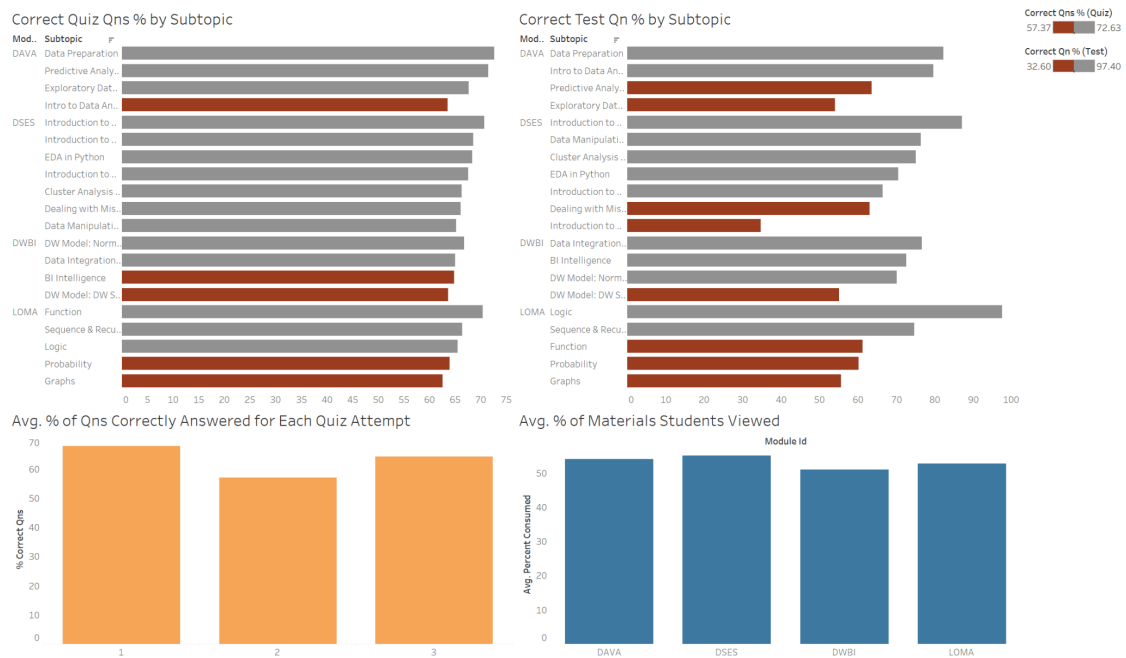


Figure 6.6: View of Lecturer Dashboard

The lecturer's dashboard is designed in a manner where lecturers are able to view the entire cohort's performance up till the granularity of subtopics. This allows for lecturers to be able to easily know the exact subtopics that most students are weaker in and prepare targeted assistance in those areas.

From the dashboard, we are able to view the subtopics that the cohort are generally weaker in for both quiz and test. Those topics with less than 65% of the questions correctly answered will be highlighted in red. From the dashboard, we can see that for quiz and test, students are weaker in subtopics 'DW Model: DW Schema' for DWBI and 'Probability' and 'Graphs' for LOMA. This highlights the need for lecturers to continue providing more assistance in these subtopics as the graph shows that the student truly does not understand these subtopics. The dashboard also shows that most students are not able to fully grasp the concepts for 'Predictive Analysis' and 'Exploratory Data' for DAVA and 'Dealing with Missing Data in Python' and 'Introduction to Regression with stats models in Python' for DSES as most students are unable to answer the questions related to these topics correctly during tests although they seem to be able to answer most questions correctly during quizzes. This highlights the need for lecturers to give more study materials on these topics and test the students more to see if they truly understand these topics in order to bridge the learning gaps.

The lecturers are also able to filter the graphs by the number of times the students attempted the quizzes. This allows lecturers to understand whether the student's overall performance for each subtopic has improved as they attempted the quizzes more or does the attempting the quizzes more not improve performance. This would allow lecturers to know how to plan future studying materials and quiz format. If performance does improve with the increase of attempts, lecturers can allow students to be able to attempt quizzes for an unlimited amount of times to allow them to better understand the concepts taught in class. If performance does not improve with the increase of attempts, lecturers can limit the number of quiz attempts to allow students to use the quiz to check their understanding and show the students the questions that they have gotten incorrect, but don't show them the correct answers. This allows students to know the areas that they are lacking in and allow for targeted revision of the topics before reattempting the quiz, effectively bridging the studying gap.

Lecturers can also view the average percentage of materials that the cohort had viewed for the respective modules, which allows them to know the reason behind poor performance in a certain module. It can be related to students not viewing enough of the lesson materials, which lecturers can take action by encouraging students to read and use more of the lecture materials provided to them or other factors such as the lesson materials not being easily understandable for the students, which is a signal to the lecturers that they need to change the lecture materials in order for students to be able to learn more effectively from it.

There are other charts that are implemented in the Streamlit web application which brings across similar points as some of the points in the dashboard above but it is more to complement the GenAI portion of the page.

Chapter 7: Evaluation/Review

Strengths and Value

Synthetic Data Generation

- **Value:** Implemented a flexible, scalable pipeline for producing realistic synthetic datasets using OpenAI GPT models and Python. This allows simulation of quiz/test data and engagement metrics without breaching privacy.
- **Impact:** Solves the challenge of data scarcity and privacy restrictions, particularly relevant for educational analytics, while allowing for development and testing of predictive models without relying on sensitive data.

Backend-as-a-Service (Supabase)

- **Value:** Provided an efficient, scalable, and secure backend to store, query, and manage synthetic data and analytics results
- **Impact:** Reduced development time by eliminating the need to build a backend from scratch, while supporting features like vector search for AI-powered retrieval.

Python Web Development (Streamlit)

- **Value:** Enabled rapid prototyping of a data-driven web application entirely in Python, reducing the need for a separate front-end interface.
- **Impact:** Allowed quick iteration of UI features, making analytics and AI tools accessible to non-technical users.

Generative AI Features

- **Value:** Goes beyond static data generation, offering interactive, personalized AI-driven feedback, flashcards, and explanations tailored to the student's performance.
- **Impact:** Increased engagement and learning effectiveness while demonstrating how AI can personalize education experience.

Data Visualization & Predictive Modelling

- **Value:** Used Tableau dashboards connected to Supabase for real-time, interactive exploration of data, and implemented K-Means clustering for student segmentation.
- **Impact:** Gave educators and students actionable insights, such as targeting weaker topics and encouraging the use of lesson materials.

Limitations and Recommendations

Some of the limitations I faced in this project involved the limited customisability of Streamlit application and the limited advanced AI functionality I was able to do within the limited amount of time I had for the project.

Limited Streamlit customisability. Streamlit's user interface renders in a sequential, top-to-bottom manner. This means that pages with many AI-powered elements take a relatively long amount of time to render.

Recommendation: Optimise rendering by breaking down complex pages into separate tabs or views, preloading frequently used data in the backend, and embedding custom HTML/CSS/JavaScript for interactive components. For production scaling, consider

integrating Streamlit with a more customizable front-end framework (e.g., React) while keeping the Python backend for data and AI processing.

Limited advanced AI functionality. I was not able to deliver more advanced Retrieval augmented generation (RAG) features due to the limited timeframe of this project.

Recommendation: Leverage the project's modular architecture to integrate advanced RAG techniques in future iterations, such as summary embeddings, windowed context retrieval, metadata filtering, and fine-tuning. This would allow the AI to provide richer, more context-aware responses while maintaining scalability and adaptability for evolving datasets.

Chapter 8: Reflection

Lessons Learnt and Insights Gained

Looking back at this project, I would say that working on this project has been tough but fruitful for me. It was a journey that was filled with challenges and lessons for both my technical and soft skills, from learning new technical tools to learning how to juggle between multiple responsibilities.

For starters, I now know the importance of defining a good data schema more impactfully after the generation of synthetic data for this project. For my past projects and assignments in school, I only needed to do preparation of the data and the data was ready to be used. But in this project, I found myself regenerating the datasets multiple times before I finally settled on the one that I was satisfied with and could generate insights with the clustering model and I only realised it was because I failed to define the data schema before I even generated the synthetic data, resulting in the repetitive generation of data. This made me realise that defining a good data schema before any generation of data can prevent time wasted on multiple data generation processes.

In this project, I was also able to pick up skills on new tools. Since this project requires the use of tools that I have not much knowledge on, I spent quite some time researching and learning about them. By the end of the project, I have learnt more about the tools and how to use them effectively to improve the scalability, automation and certain features of my project. For example, with the use of Supabase, users can upload their CSV files directly to the cloud through the application and the data from the CSV file is stored in the database, and any updates made through the application is reflected instantly in Supabase. This reduces the

need for manual re-uploads or refreshes when changes is made to the data, improving efficiency and scalability.

Since I had to complete this project alongside my electives, it really trained my time management skills. There was a particular week where I had to submit both my elective assignments and work on the project at the same time, I had to juggle between both assignments and the project. I ended up pulling a few all-nighters that week to finish those assignments and make progress on the project at the same time because of my poor time management and planning. After this stressful event, I started to pay more attention to make more wiseful planning of my time and prioritize tasks that are due first/are of more importance.

References/ Bibliography

- Akkem, Y., Kumar, B. S., & Varanasi, A. (2023, December 29). Streamlit application for advanced ensemble learning methods in crop recommendation systems – A review and implementation. *Indian Journal of Science and Technology*, 16(48).
<https://doi.org/10.17485/IJST/v16i48.2850>
- Allied Market Research. (2023, December). *Backend-as-a-Service (BaaS) market size, share, competitive landscape and trend analysis report, by service type ... Global opportunity analysis and industry forecast, 2022–2032*.
<https://www.alliedmarketresearch.com/backend-as-a-service-market>
- Amazon Web Services. (n.d.). AWS Amplify. <https://aws.amazon.com/amplify/>
- Andrews, K. (n.d.). *Using Docker Compose for Python development*. CloudBees.
<https://www.cloudbees.com/blog/using-docker-compose-for-python-development>
- Blumenstyk, G. (2018, April 8). Can artificial intelligence make teaching more personal? *The Chronicle of Higher Education*.
<https://www.chronicle.com/article/can-artificial-intelligence-make-teaching-more-personal/>
- Brempong, O. W. J. (2021, October). K-means clustering algorithm for student's selection and performance prediction. *International Journal of Scientific & Technology Research*, 10(10).
<https://www.ijstr.org/final-print/oct2021/K-means-Clustering-Algorithm-For-Students-Selection-And-Performance-Prediction.pdf>
- Caffagni, D., Cocchi, F., Barsellotti, L., Moratelli, N., Sarto, S., Baraldi, L., Cornia, M., & Cucchiara, R. (2024, June 6). *The revolution of multimodal large language models: A survey*. arXiv. <https://arxiv.org/pdf/2402.12451>
- Cloudflare. (n.d.). *What is BaaS? | Backend-as-a-Service vs. serverless*.
<https://www.cloudflare.com/en-gb/learning/serverless/glossary/backend-as-a-service-baas/>
- Clow, D. (2012). The learning analytics cycle: Closing the loop effectively. In *Proceedings of the 2nd International Conference on Learning Analytics and Knowledge (LAK '12)* (pp. 134–138). ACM.
<https://scispace.com/pdf/the-learning-analytics-cycle-closing-the-loop-effectively-4pnc02kuaz.pdf>

- CodeDealer. (n.d.). *Streamlit Web UI interface*.
<https://codedealer.github.io/stable-diffusion-webui/docs/4.streamlit-interface.html>
- DataCamp. (2024, January 12). *Top 26 Python libraries for data science in 2025*.
<https://www.datacamp.com/blog/top-python-libraries-for-data-science>
- FFmpeg. (n.d.). *HWAccelIntro*. <https://trac.ffmpeg.org/wiki/HWAccelIntro>
- Flask. (n.d.). *Flask*. <https://flask.palletsprojects.com/en/stable/>
- Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., & Wang, H. (2023, December 18). Retrieval-augmented generation for large language models: A survey. Beijing Academy of Artificial Intelligence.
<https://simg.baai.ac.cn/paperfile/25a43194-c74c-4cd3-b60f-0a1f27f8b8af.pdf>
- Goodfellow, I. J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014, June 10). *Generative adversarial nets*. arXiv.
<https://arxiv.org/pdf/1406.2661>
- Goyal, M., & Mahmoud, Q. H. (2024). A systematic review of synthetic data generation techniques using generative AI. *Electronics*, 13(17), 3509.
<https://doi.org/10.3390/electronics13173509>
- Gupta, V. (2024). *Building an MVP app using Supabase*. Medium.
<https://medium.com/@antstack/building-an-mvp-app-using-supabase-2961ea390570>
- Halper, D. (2024, October 8). 'Learn faster': Duke student startup TurboLearn AI seeks to bring AI into the classroom. *The Chronicle*.
<https://www.dukechronicle.com/article/2024/10/duke-university-turbo-learn-ai-duke-student-start-up-sarthak-dhawan-artificial-intelligence>
- Herodotou, C., Maguire, C., McDowell, N., Hlostá, M., & Boroowa, A. (2021, July 12). The engagement of university teachers with predictive learning analytics. *Computers & Education*. <https://doi.org/10.1016/j.compedu.2021.104294>
- IBM. (2024, October 14). *AI and the future of unstructured data*.
<https://www.ibm.com/think/insights/unstructured-data-trends>
- Jävergård, N., Muntean, A., Lyons, R., & Forsman, J. (2025, June 24). *Tunable correlation retention: A statistical method for generating synthetic data*. arXiv.
<https://arxiv.org/pdf/2403.01471>
- Kapoor, A., & Kumar, M. (2025, May 8). Generative AI and personalized video advertisements. *Marketing Science*. <https://doi.org/10.1287/mksc.2023.0494>

- Khan, A. (n.d.). *How to make better Anki flashcards: Principles for high-quality questions*. LEANANKI. <https://leananki.com/creating-better-flashcards>
- Lambda Solutions. (n.d.). *Understanding the purpose of a learning management system (LMS)*. <https://www.lambdasolutions.net/en/blog/understanding-the-purpose-of-a-learning-management-system-lms#>:
- Lang, C., Wise, A. F., Merceron, A., & Gasevic, D. (2022, January). What is learning analytics. In C. Lang, A. F. Wise, A. Merceron, & D. Gasevic (Eds.), *The handbook of learning analytics* (pp. 8–18). <https://doi.org/10.18608/hla22.001>
- LangChain. (2023, October 20). *Multi-vector retriever for RAG on tables, text, and images*. <https://blog.langchain.com/semi-structured-multi-modal-rag/>
- Lei, K., Ma, Y., & Tan, Z. (2014, December). Performance comparison and evaluation of web development technologies in PHP, Python, and Node.js. In *2014 IEEE 17th International Conference on Computational Science and Engineering* (pp. 661–668). IEEE. <https://doi.org/10.1109/CSE.2014.142>
- Li, T., Fan, Y., Tan, Y., Wang, Y., Singh, S., Li, X., Raković, M., van der Graaf, J., Lim, L., Yang, B., Molenaar, I., Bannert, M., Moore, J., Swiecki, Z., Tsai, Y.-S., Shaffer, D. W., & Gašević, D. (2023, August 3). Analytics of self-regulated learning scaffolding: Effects on learning processes. *Frontiers in Psychology*, 14. <https://doi.org/10.3389/fpsyg.2023.1206696>
- Li, Z., Wang, Z., Wang, W., Hung, K., Xie, H., & Wang, F. L. (2025, May 14). Retrieval-augmented generation for educational application: A systematic survey. *Computers and Education: Artificial Intelligence*. <https://www.sciencedirect.com/science/article/pii/S2666920X25000578>
- Lu, Y., Chen, L., Zhang, Y., Shen, M., Wang, H., Wang, X., van Rechem, C., Fu, T., & Wei, W. (2025, June 4). *Machine learning for synthetic data generation: A review*. arXiv. <https://arxiv.org/pdf/2302.04062>
- MDN Web Docs. (n.d.). *JavaScript frameworks and libraries*. https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Frameworks_libraries
- OpenAI. (2023, December 4). *Semantic search using Supabase Vector*. OpenAI Cookbook. https://cookbook.openai.com/examples/vector_databases/supabase/semantic-search

- OpenAI. (2025, April 16). *Introducing OpenAI o3 and o4-mini*.
<https://openai.com/index/introducing-o3-and-o4-mini/>
- OpenAI. (2025, February 2). *Introducing deep research*.
<https://openai.com/index/introducing-deep-research/>
- OpenAI. (2025). *Responses*. <https://platform.openai.com/docs/api-reference/responses>
- OpenAI. (n.d.). *Models*. OpenAI Platform. <https://platform.openai.com/docs/models>
- Pass, J. C. (2024, June 7). Exploring the effectiveness of flashcards for learning and retention. *Simply Put Psych*.
<https://simplyputpsych.co.uk/psych-101-1/exploring-the-effectiveness-of-flashcards-for-learning-and-retention>
- Personal Data Protection Commission. (2024). *Privacy enhancing technology (PET): Proposed guide on synthetic data generation*.
<https://www.pdpc.gov.sg/-/media/files/pdpc/pdf-files/other-guides/proposed-guide-on-synthetic-data-generation.pdf>
- Qin, Z., Han, X., Sun, W., Li, D., Kong, L., Barnes, N., & Zhong, Y. (2022, October 19). The devil in linear transformer. arXiv. <https://arxiv.org/pdf/2210.10340>
- Radford, A., & Narasimhan, K. (2018). *Improving language understanding by generative pre-training*. OpenAI.
<https://www.cs.ubc.ca/~amuham01/LING530/papers/radford2018improving.pdf>
- Raschka, S. (2025, February 5). Understanding reasoning LLMs: Methods and strategies for building and refining reasoning models.
<https://magazine.sebastianraschka.com/p/understanding-reasoning-llms>
- Richardson, G. (n.d.). *Semantic search using Supabase Vector*. OpenAI Cookbook.
https://cookbook.openai.com/examples/vector_databases/supabase/semantic-search
- Roos, P. (2022). *Under the hood: Architecture and technology stack of Supabase*.
<https://www.workingsoftware.dev/tech-stack-and-architecture-of-supabase>
- Salas-Pilco, S. Z., Xiao, K., & Hu, X. (2022). Artificial intelligence and learning analytics in teacher education: A systematic review. *Education Sciences*, 12(8), 569.
<https://doi.org/10.3390/educsci12080569>
- SAP. (n.d.). *What is a learning management system (LMS)?*.
<https://www.sap.com/sea/products/hcm/corporate-lms/what-is-lms.html>
- Siemens, G., Gasevic, D., Haythornthwaite, C., Dawson, S., Shum, S. B., Ferguson, R., Duval, E., Verbert, K., & Baker, R. S. J. D. (2011, July 28). *Open learning analytics:*

- An integrated & modularized platform*. Society for Learning Analytics Research.
<https://solaresearch.org/wp-content/uploads/2011/12/OpenLearningAnalytics.pdf>
- Streamlit. (n.d.). *Streamlit*. <https://streamlit.io/>
- Tableau. (n.d.). *Tableau*. <https://www.tableau.com/>
- University of Bath. (2021, April 22). Descriptive, predictive and prescriptive: Three types of business analytics.
<https://online.bath.ac.uk/content/descriptive-predictive-and-prescriptive-three-types-business-analytics>
- Uunonen, S. (2023). *Backend as a service in web development* (Bachelor's thesis). Theseus.
https://www.theseus.fi/bitstream/handle/10024/801937/Uunonen_Samu.pdf?sequence=2
- Wu, S., Xiong, Y., Cui, Y., Wu, H., Chen, C., Yuan, Y., Huang, L., Liu, X., Kuo, T.-W., Guan, N., & Xue, C. J. (2025, March 11). Retrieval-augmented generation for natural language processing: A survey. arXiv. <https://arxiv.org/pdf/2407.13193>
- Yan, L., Martinez-Maldonado, R., & Gasevic, D. (2024, March 18). Generative artificial intelligence in learning analytics: Contextualising opportunities and challenges through the learning analytics cycle. In *Proceedings of the 14th Learning Analytics and Knowledge Conference (LAK '24)* (pp. 101–111). ACM.
<https://doi.org/10.1145/3636555.3636856>
- Yosifova, A. (2024, October 10). The data scientist job market in 2024 [Research on 1,000 job postings]. 365 *Data Science*.
<https://365datascience.com/career-advice/data-scientist-job-market/>

Appendix A1: Terms of Reference

Project Particulars

Organization Name	School of Informatics and IT (Temasek Polytechnic)
MP Supervisor	Ms Lee Esther
Project Title	AI-Enhanced Learning Insights and Personalized Student Support
Student Matric Card Number	2300984H
Student Name	Sing Yan Ling

1. Introduction

Learning Management System (LMS) is an online platform where students can interact with various online learning materials such as quizzes, e-lectures and study material uploaded by their lecturers. However, these resources are spread across the website, lacking an integrated view and lecturers rely on manual tracking methods to identify at-risk students and weak topic areas, leading to inefficiencies and difficulty in monitoring student progress and performance. This project aims to develop a unified analytics platform that provides real-time insights into student engagement and performance through integrated dashboards and intelligent feedback mechanisms.

2. Objectives

Objectives of this project is to:

- Develop a unified platform that provides real-time visibility into student engagement and performance across the learning materials (quizzes, videos, tests)

- Automatically identify common mistakes and weak areas/topics students have using data analytics and AI
- Provide personalized learning recommendations tailored to each student's performance and engagement patterns
- Implement an AI-powered explanation system to help students learn, understand their mistakes and reinforce their learning
- Reduce manual and time-consuming academic monitoring by enabling automated dashboards and intervention triggers for lecturers

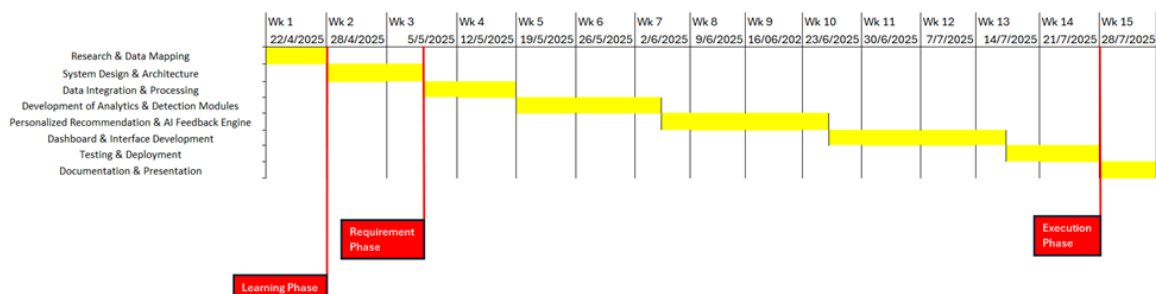
3. Scope of the Project

- Develop interactive dashboards for lecturers and students to track performance and progress
- Set up automated data pipelines to integrate, preprocess and store data from various learning materials
- Build machine learning models to detect common mistakes, identify weak topics and predict students at-risk of poor performance
- Implement AI-powered algorithms to explain student errors and recommend personalized study materials based on individual weak areas

4. Project Plan

Task no	Description	Start Date	End Date	Duration
1	Research & Data Mapping: - Generate synthetic dataset & define data schema - Do initial check for any data quality issues - Understand requirements for the dashboard (what metrics needs to be on the dashboard) - Identify key metrics to use in dashboard	22 April 2025	27 April 2025	1 Week
2	System Design & Architecture: - Design system components and data flow - Map out the tech stack	28 April 2025	7 May 2025	1.5 Weeks
3	Data Integration & Processing: - Build data processing pipelines to preprocess and integrate data - Set up initial data validation checks to ensure data quality	8 May 2025	17 May 2025	1.5 Weeks

4	Development of Analytics & Detection Modules: - Build ML models to identify common areas for mistakes, weak topics, engagement patterns, students at risk etc.	18 May 2025	3 June 2025	2.5 Weeks
5	Personalized Recommendation & AI Feedback Engine: - Train models to recommend learning resources based on weak topics/topics with high amt of mistakes - Integrate external content for learning resource recommendation (e.g. YouTube videos, LinkedIn lessons) - Use LLMs (e.g. OpenAI) to explain incorrect answers in simple terms - Personalize feedback per student profile	4 June 2025	24 June 2025	3 Weeks
6	Dashboard & Interface Development: - Design & develop web dashboards - Build separate dashboards for students and lecturers - Integrate necessary metrics for each dashboards	25 June 2025	15 July 2025	3 Weeks
7	Testing & Deployment: - Test and ensure data pipeline & ML models are working as intended - Test AI feedback quality & recommendation accuracy - Use metrics like accuracy, precision and recall assessing model performance - Check for over/underfitting issues if any - Double check for any data quality issues (missing values, duplicates etc)	16 July 2025	25 July 2025	1.5 Week
8	Documentation	26 July 2025	1 Aug 2025	1 Week



Appendix A2: Weekly Progress Report (for 12 weeks)

Organization Name: Temasek Polytechnic	
Project Title: AI-Enhanced Learning Insights and Personalized Student Support	
Student Name: Sing Yan Ling	Admin No: 2300984H
Temasek Polytechnic's Supervisor Name: Ms. Esther Lee	Total No. of Weeks: 12

Individual Task Status (Report #1) (21 Apr - 27 Apr)

Task no	Description	Start Date	End Date	Status	Remarks
1	Understand requirements for the dashboard	22/04	22/04	Green	
2	Identify/Research on key metrics to be used/shown on dashboard	22/04	24/04	Green	
3	Generate Synthetic Data	24/04	25/04	Green	
4	Define Data Schema	25/04	25/04	Green	
5	Do Initial Check for any Data Quality Issues	26/04	27/04	Green	

Individual Task Status (Report #2) (28 Apr - 4 May)

Task no	Description	Start Date	End Date	Status	Remarks
1	Design System Components & Data Flow	28/04	30/04	Green	
2	Map out Tech Stack	01/05	04/05	Green	

Individual Task Status (Report #3) (5 May - 11 May)

Task no	Description	Start Date	End Date	Status	Remarks
1	Map out Tech Stack	05/05	05/05	Green	Ahead of time
2	Build pipeline to preprocess and integrate data	06/05	11/05	Green	

Individual Task Status (Report #4) (12 May - 18 May)

Task no	Description	Start Date	End Date	Status	Remarks
1	Do initial data validation checks to ensure data quality	12/05	14/05	Green	Ahead of time
2	Build ML model to identify students with common weak topics	15/05	18/05	Green	

Individual Task Status (Report #5) (19 May - 25 May)

Task no	Description	Start Date	End Date	Status	Remarks
1	Build ML model to identify students with common weak topics	19/05	21/05	Green	
2	Build ML model to identify students at risk	21/05	25/05	-	Will not be using in the project (no useful insights, using another method to identify students at risk)

Individual Task Status (Report #6) (26 May - 1 June)

Task no	Description	Start Date	End Date	Status	Remarks
1	Calculated formula to identify student at risk	26/05	27/05	Green	
2	Build LLM model to explain incorrect answers	27/05	01/06	Green	

Individual Task Status (Report #7) (2 June - 8 June)

Task no	Description	Start Date	End Date	Status	Remarks
1	Build LLM model to explain incorrect answers	02/06	08/06	Green	

Individual Task Status (Report #8) (9 June - 15 June)

Task no	Description	Start Date	End Date	Status	Remarks
1	Personalize feedback per student profile	09/06	15/06	Green	

Individual Task Status (Report #9) (30 June - 6 July)

Task no	Description	Start Date	End Date	Status	Remarks
1	Build student's dashboard on Tableau	30/06	30/06	Green	
2	Design & build teacher's dashboard on Tableau	01/07	06/07	Green	

Individual Task Status (Report #10) (7 July - 13 July)

Task no	Description	Start Date	End Date	Status	Remarks
1	Build teacher's dashboard on Tableau	07/07	08/07	Green	
2	Develop web to integrate GenAI and dashboard into consolidated platform	09/07	13/07	Green	

Individual Task Status (Report #11) (14 July - 20 July)

Task no	Description	Start Date	End Date	Status	Remarks
1	Develop web to integrate GenAI and dashboard into consolidated platform	14/07	20/07	Yellow	

Individual Task Status (Report #12) (21 July - 27 July)

Task no	Description	Start Date	End Date	Status	Remarks
1	Test AI feedback quality & double check data quality issues	21/07	25/07	Green	
2	Report Documentation	26/07	08/07	Green	